Report Title: Comparative Analysis of Statistical and Deep Learning Approaches for Emotion Recognition in Text

This project investigates the effectiveness of Natural Language Processing (NLP) techniques for multi-class emotion recognition from textual data. The study is based on a large-scale, real-world dataset sourced from **Kaggle**, commonly referred to as the *Emotions Dataset*. The dataset consists of **416,809 English-language text samples**, primarily collected from the Twitter (now X) platform, making it representative of informal, user-generated social media content. Each text entry is annotated with one of **six discrete emotion labels**: *Joy, Sadness, Anger, Fear, Love,* and *Surprise*. The dataset is provided in a structured CSV format, with raw text paired to a single categorical label, enabling direct application of supervised machine learning techniques.

The size and diversity of the dataset make it particularly suitable for evaluating both traditional statistical classifiers and modern deep learning architectures. However, its real-world nature also introduces challenges such as informal grammar, abbreviations, slang, emotive exaggeration, and class imbalance, all of which are explicitly addressed within the project's preprocessing and evaluation strategy. To assess how different modeling paradigms handle these challenges, four classifiers were implemented and compared: two statistical baselines—Multinomial Naive Bayes and Linear Support Vector Machine (SVM)—and two neural approaches—a Feed-Forward Neural Network and a Bi-Directional Long Short-Term Memory (LSTM) network incorporating pre-trained GloVe word embeddings. The investigation places particular emphasis on the role of semantic preprocessing and word embeddings in improving contextual understanding. Experimental results demonstrate that while statistical models offer strong computational efficiency and reasonable baseline performance, deep learning architectures—especially those leveraging pre-trained embeddings—are more effective at capturing contextual nuance and sequential dependencies inherent in informal social media text.

1. Introduction

1.1 Domain-Specific Area

The rapid growth of social media platforms has resulted in an unprecedented volume of unstructured textual data, providing a rich and continuous stream of user-generated content. This data offers valuable insight into public sentiment, emotional expression, and psychological states at both individual and collective levels. Within the broader field of Natural Language Processing (NLP), this project is situated in the domain of **Emotion Recognition**, which is a specialized and more granular extension of traditional sentiment analysis. While conventional sentiment analysis typically categorizes text into coarse classes such as positive, negative, or neutral, emotion recognition seeks to identify **specific emotional states**expressed within text, enabling deeper semantic and contextual understanding.

This project focuses on the automatic classification of text into six discrete emotional categories: **Joy, Sadness, Anger, Fear, Love, and Surprise**. These classes go beyond simple polarity and capture a wider range of human affect, including both positive and negative emotions as well as more ambiguous or context-dependent states. Identifying such fine-grained emotions presents

significant computational challenges, particularly when working with social media data. Text originating from platforms such as Twitter is often informal and noisy, containing slang, abbreviations, emotive exaggeration, hashtags, emojis, and non-standard grammar. Additionally, emotions are frequently implied rather than explicitly stated, and linguistic phenomena such as negation, sarcasm, and semantic overlap between emotions further complicate classification.

From a modeling perspective, these characteristics place high demands on feature representation and contextual understanding. Bag-of-words-based statistical models may struggle to capture sequential dependencies and nuanced meaning, whereas neural architectures capable of modeling word order and semantic relationships offer potential advantages. As a result, emotion recognition serves as an appropriate and challenging testbed for comparing traditional statistical NLP approaches with modern deep learning techniques, which is a central aim of this coursework.

Despite these challenges, the practical applications of emotion recognition are substantial. In the domain of **mental health monitoring**, automated analysis of emotional patterns in social media posts can assist in identifying sustained expressions of sadness, fear, or distress, potentially enabling early intervention or support. In **customer experience and brand management**, emotion-aware systems allow organizations to distinguish between urgent negative feedback (such as anger-driven complaints) and positive engagement, enabling more effective prioritization than simple sentiment scoring. By addressing the technical difficulties inherent in social media text and evaluating multiple modeling strategies, this project contributes to understanding how different NLP techniques perform in a realistic, high-noise, multi-class emotion recognition setting.

1.2 Objectives

The primary objective of this coursework is to conduct a structured and empirical comparison between **traditional statistical Natural Language Processing techniques** and **modern deep learning architectures** for the task of multi-class emotion recognition from text. By implementing and evaluating multiple modeling approaches on the same large-scale social media dataset, the project aims to assess how differences in feature representation, model capacity, and contextual awareness affect classification performance in a realistic, high-noise environment.

A central objective of the project is the development of a **robust text preprocessing pipeline** tailored to the characteristics of social media data. Given the informal nature of tweets, the preprocessing stage is designed to reduce noise while preserving emotion-bearing linguistic cues. This includes normalization steps such as lowercasing, removal of irrelevant artifacts (e.g. URLs and user mentions), **contraction expansion** to improve negation handling, **lemmatization**to reduce morphological variation, and **stopword removal** to limit the influence of semantically weak tokens. These preprocessing decisions are explicitly implemented and verified within the code to ensure that downstream models are trained on cleaner and more informative textual representations.

Another key objective is to establish strong **statistical baselines** using well-understood and widely adopted classifiers. To this end, **Multinomial Naive Bayes** and **Linear Support Vector Machines (SVM)** are implemented in combination with **TF-IDF vectorization**, representing the traditional "bag-of-words" paradigm in text classification. These models provide a computationally efficient reference point against which the performance gains of more complex neural models can be meaningfully assessed.

In parallel, the project aims to implement and evaluate **deep learning architectures** using the TensorFlow/Keras framework. Two neural models are developed: a **Feed-Forward Neural Network**, serving as a deep learning baseline, and a more advanced **Bi-Directional Long Short-Term Memory (LSTM)** network. The LSTM model incorporates **pre-trained GloVe word**

embeddings, enabling the network to leverage external semantic knowledge and capture sequential dependencies and contextual relationships between words—capabilities that are not available to statistical models.

Finally, the project aims to **systematically evaluate and compare all models** using a consistent set of performance metrics. Classification performance is assessed using **Accuracy** as a high-level baseline metric, alongside **F1-Score** to account for class imbalance within the dataset. In addition, **Confusion Matrices** are employed to perform detailed error analysis, enabling examination of class-specific performance and common misclassification patterns. Through this multi-metric evaluation strategy, the project seeks to determine which modeling approach is most effective at capturing the nuanced emotional context present in informal social media text.

1.3 Dataset Description

The dataset used in this study is the **Emotions Dataset**, obtained from the **Kaggle open data platform**. Provided in CSV format, it contains **416,809 English-language text samples**, making it a large-scale corpus suitable for supervised machine learning. The data is primarily sourced from **Twitter (now X)** and therefore reflects informal, user-generated social media text rather than curated or professionally edited content.

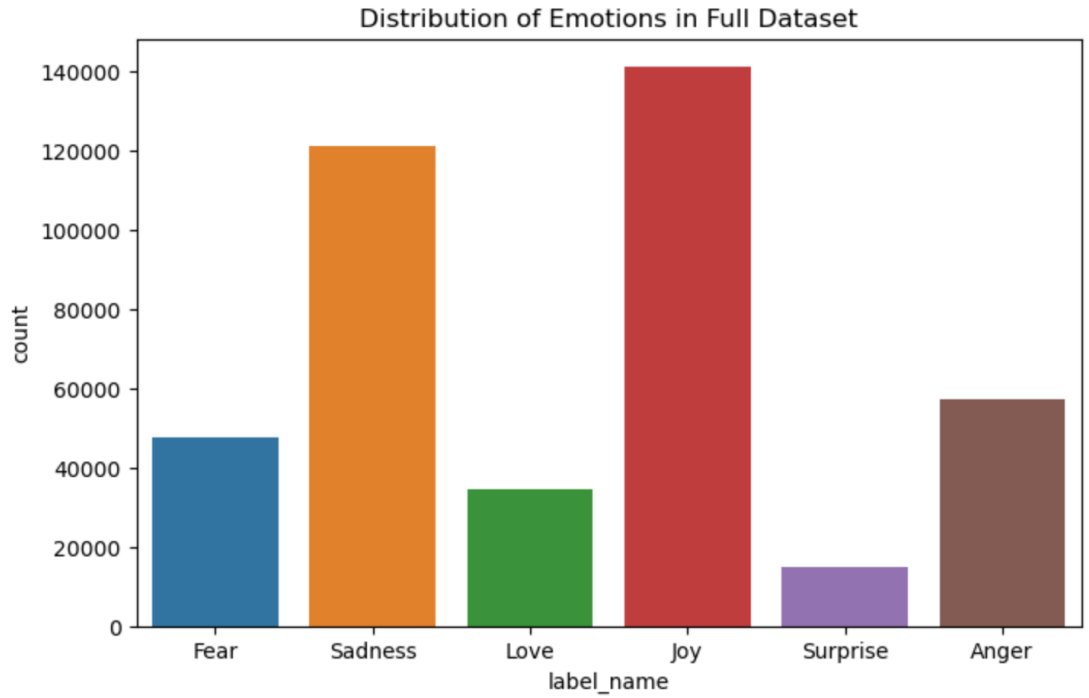
The dataset's size and diversity are key strengths, supporting robust model training and reducing overfitting risk, particularly for deep learning approaches. At the same time, it reflects real-world linguistic variability—such as slang, abbreviations, emotive emphasis, hashtags, and irregular punctuation—introducing substantial noise and creating a realistic and challenging NLP classification task aligned with practical applications rather than simplified academic benchmarks.

Each text instance is annotated with a **single categorical target label** representing the dominant emotion expressed in the text. This framing allows the problem to be treated as a **multi-class supervised classification task**, where the objective is to assign each input text to exactly one emotional category. The dataset contains six distinct emotion classes: **Joy (Positive), Sadness (Negative), Anger (Negative), Fear (Negative), Love (Positive), and Surprise (Neutral/ Ambiguous)**. These categories extend beyond binary sentiment polarity and require models to capture nuanced semantic and contextual cues. Certain classes, such as *Joy* and *Love*, share overlapping positive vocabulary, while others, such as *Fear* and *Surprise*, are highly context-dependent. This overlap increases task difficulty and makes the dataset well suited for evaluating both statistical feature-based models and neural sequence-based approaches.

An initial **exploratory data analysis (EDA)** revealed a clear **class imbalance** across the six emotion categories. The class distribution visualization shows that *Joy* and *Sadness* account for a disproportionately large share of the dataset, while *Surprise* and *Love* are significantly underrepresented. This imbalance reflects natural patterns of emotional expression on social media, where certain emotions are expressed more frequently and explicitly than others.

```
Total Dataset Size: 416809
```

	text	label	label_name
0	i just feel really helpless and heavy hearted	4	Fear
1	ive enjoyed being able to slouch about relax a...	0	Sadness
2	i gave up my internship with the dmrg and am f...	4	Fear
3	i dont know i feel so lost	0	Sadness
4	i am a kindergarten teacher and i am thoroughl...	4	Fear



Class imbalance has important implications for both model training and evaluation. Models trained on imbalanced data are inherently biased toward majority classes, as they are exposed to a greater number of examples from those categories. As a result, a naive classifier that always predicts the most frequent class—*Joy*—could achieve deceptively high accuracy while failing to detect minority emotions. Such behavior would be unacceptable in practical emotion recognition systems, where accurately identifying less frequent but semantically important emotions, such as *Surprise* or *Fear*, is often critical. Consequently, accuracy alone is insufficient as an evaluation metric, and the **F1-score** is adopted as a primary measure to ensure balanced assessment across all classes.

To ensure a valid assessment of model generalization, the full dataset was split into **training (80%) and testing (20%) subsets**. A **stratified splitting strategy** was employed to preserve the original class distribution across both sets. Without stratification, random splitting could lead to test data that underrepresents or omits minority classes, undermining evaluation validity. Stratified sampling ensures that all six emotions are consistently represented during both training and evaluation, enabling fair and reliable comparison of model performance.

```
# Split features and labels into training and testing partitions
# Training receives 80% of samples, testing receives 20% for evaluation
# Stratification preserves label distribution across both splits
X_train, X_test, y_train, y_test = train_test_split(
    df['clean_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']  # Keeps class proportions consistent in train and test sets
)
```

```
Cleaning text... this might take a moment.
Data successfully cleaned and split!
Training Data: 333447 rows
Testing Data: 83362 rows
```

1.4 Evaluation Methodology

To ensure a fair comparison of the four classification models—Multinomial Naive Bayes, Linear SVM, Feed-Forward Neural Network, and Bi-Directional LSTM—a consistent evaluation strategy was employed. All models were trained on the same stratified training set and evaluated on an identical held-out test set, ensuring that performance differences reflect modeling choices rather than data variation, and enabling meaningful comparison between statistical and deep learning approaches.

- Performance Metrics

Given the class imbalance identified during exploratory analysis, relying on a single evaluation metric would be misleading. Accuracy alone can overestimate performance in imbalanced multi-class settings by favoring majority classes, so a suite of complementary evaluation metrics was employed.

Accuracy was retained as a high-level baseline metric, measuring the proportion of correct predictions across the test set. While useful for summarizing overall correctness, it does not capture class-wise performance disparities. To provide deeper insight, **macro-averaged Precision and Recall** were computed: precision measures the reliability of predicted emotion labels, while recall assesses how effectively the model identifies all instances of each emotion. Macro-averaging ensures equal contribution from both majority and minority classes.

The **Weighted F1-Score** was selected as the **primary metric for model comparison**. As the harmonic mean of precision and recall, the F1-score balances false positives and false negatives. The weighted variant further accounts for class support while penalizing poor performance on underrepresented classes such as *Surprise*. This makes it particularly well suited for evaluating emotion recognition on imbalanced social media data. Collectively, this multi-metric strategy ensures that performance is assessed in terms of both overall correctness and class-specific robustness.

```
--- EVALUATING STATISTICAL MODEL (Naive Bayes) ---

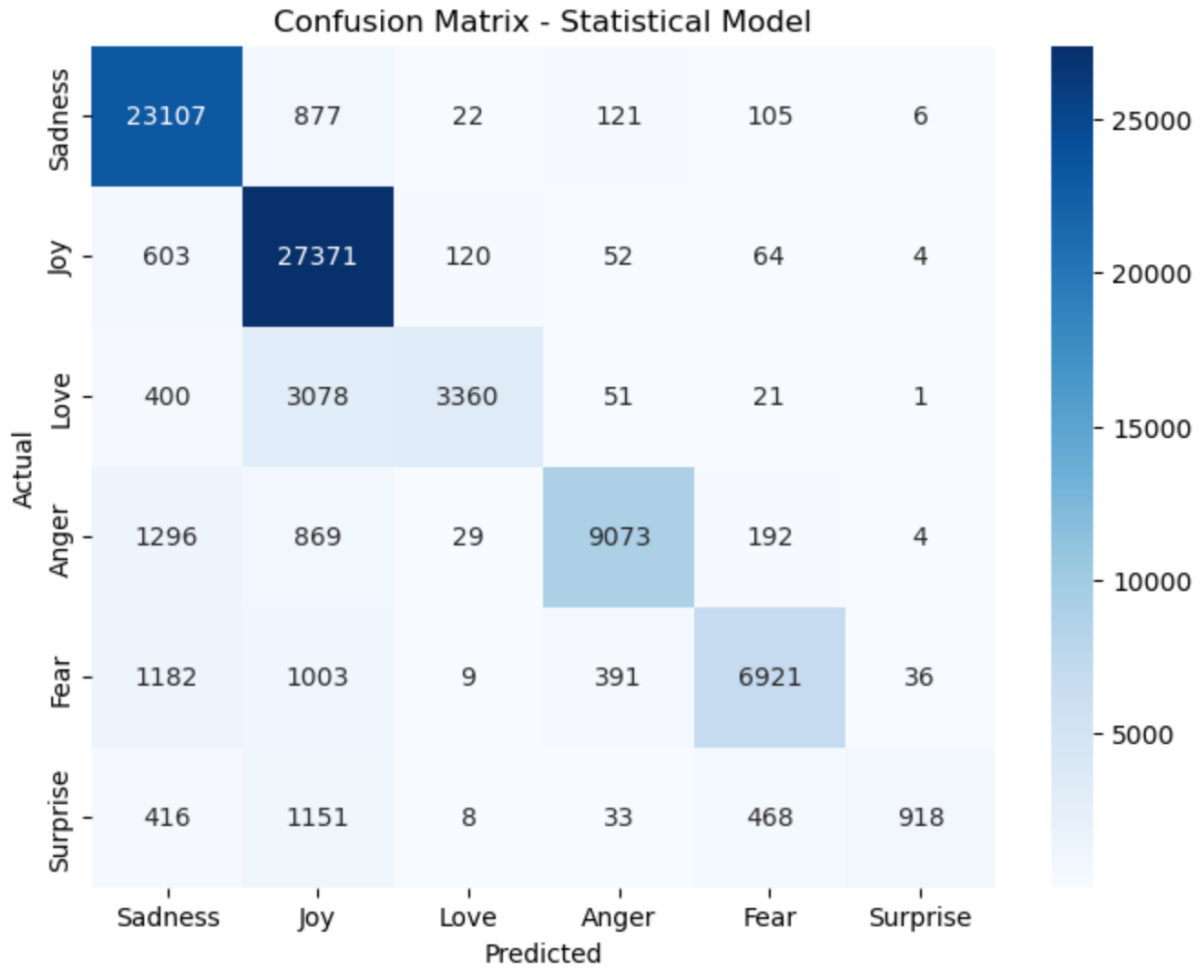
Classification Report:
              precision    recall  f1-score   support

     Sadness       0.86      0.95      0.90     24238
         Joy       0.80      0.97      0.87     28214
        Love       0.95      0.49      0.64      6911
       Anger       0.93      0.79      0.86     11463
        Fear       0.89      0.73      0.80      9542
    Surprise       0.95      0.31      0.46      2994

    accuracy                           0.85     83362
   macro avg       0.90      0.71      0.76     83362
weighted avg       0.86      0.85      0.84     83362
```

- Confusion Matrices

To complement scalar metrics, confusion matrices were generated for detailed error analysis, providing a class-by-class breakdown of predictions that reveals patterns not captured by aggregate scores alone. The matrices show consistent confusion between semantically related emotions, such as *Love* being misclassified as *Joy* and *Anger* as *Sadness*, reflecting overlapping vocabulary and contextual usage. They also highlight class imbalance effects, with minority emotions like *Surprise* exhibiting lower recall. Models with stronger contextual modeling, particularly the Bi-Directional LSTM, demonstrate improved discrimination for these challenging classes.



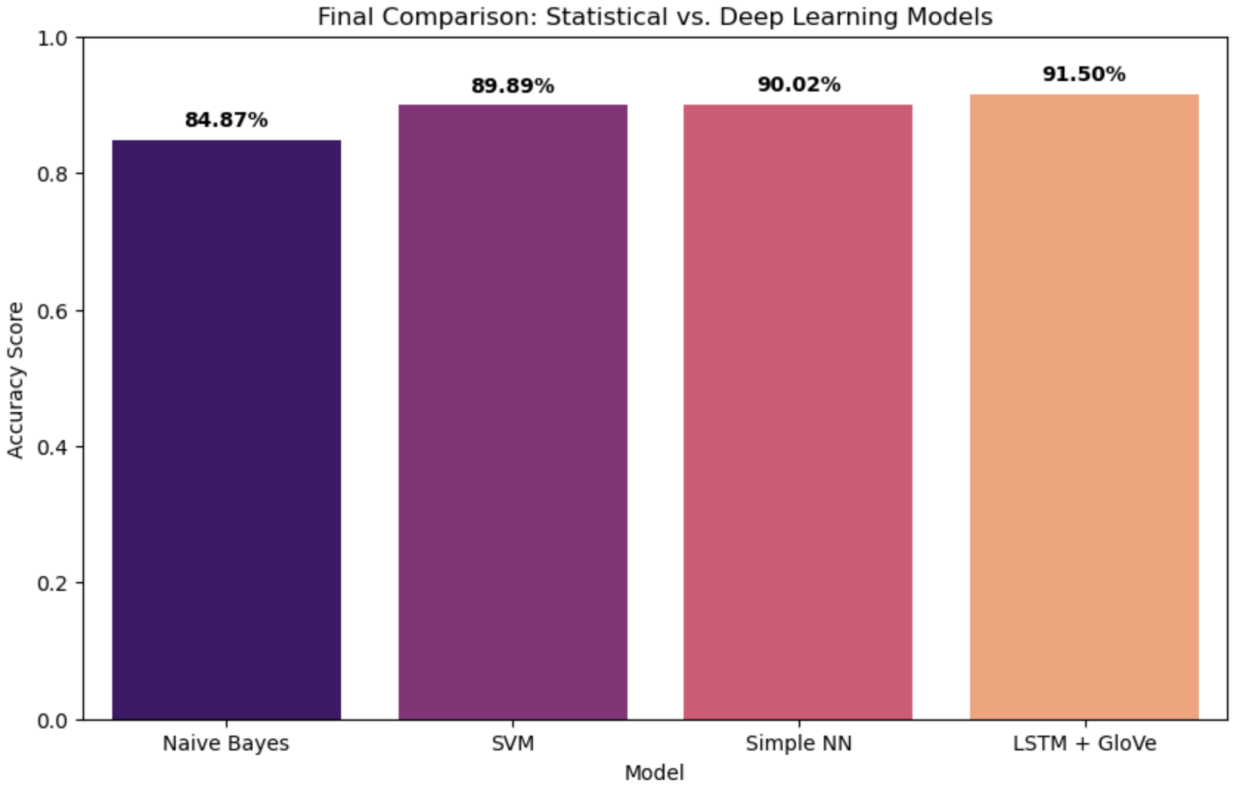
- Baseline Comparison and Evaluation

The evaluation adopts a **progressive complexity framework**, comparing increasingly sophisticated models to assess the value added by each approach. The statistical baselines—**Multinomial Naive Bayes and Linear SVM**—use **TF-IDF representations** and reflect traditional bag-of-words methodologies. Naive Bayes provides a lightweight probabilistic benchmark, while the Linear SVM serves as a strong discriminative baseline for high-dimensional sparse data.

The challenger models—the **Feed-Forward Neural Network** and **Bi-Directional LSTM with GloVe embeddings**—represent modern deep learning approaches. These models learn non-linear feature interactions, and in the case of the LSTM, explicitly model word order and contextual

dependencies. Performance gains are evaluated relative to the strong SVM baseline, particularly for context-dependent and underrepresented emotion classes, providing a principled assessment of whether increased model complexity yields meaningful improvements.

```
--- FINAL PROJECT COMPARISON ---
Naive Bayes Accuracy:      84.87%
SVM Accuracy:              89.89%
Simple Neural Net Acc:     90.02%
GloVe + LSTM Accuracy:     91.50%
```



2. Implementation

Whole of the code is added in the appendix at the last of the report.

2.1 Data Preprocessing

Text sourced from social media platforms presents unique challenges for Natural Language Processing tasks due to its informal and highly variable nature. Unlike structured or professionally edited text, social media content frequently contains slang, abbreviations, emotive exaggeration, inconsistent capitalization, non-standard spelling, and platform-specific artifacts such as hashtags, user mentions, and embedded URLs. These characteristics introduce substantial noise, which can obscure the emotional signal that classification models are intended to learn. As a result, effective preprocessing is a critical prerequisite for reliable emotion recognition.

To address these challenges, a **rigorous and deliberately designed "Super Advanced" preprocessing pipeline** was implemented as part of this project. The primary objective of this pipeline is to **maximize the signal-to-noise ratio** in the dataset by reducing irrelevant textual artifacts while preserving words and structures that convey emotional meaning. All preprocessing steps were applied consistently across the dataset prior to model training, ensuring that differences

in performance between models are attributable to modeling choices rather than inconsistencies in data preparation.

- Text Cleaning and Normalization

The initial stage of preprocessing focused on **standardizing the raw text structure** to reduce noise and improve downstream model learning. To achieve this, a custom preprocessing function, `super_clean_text`, was implemented to apply a sequence of normalization operations consistently across the dataset.

First, **HTML decoding** was applied using the `html.unescape` utility to convert encoded entities (such as `&`) back into their original character forms. This step ensures that text originating from web-based sources is restored to a readable and linguistically meaningful format.

Next, **lowercasing** was performed on all text samples. This normalization step prevents the creation of redundant vocabulary entries arising from capitalization differences (e.g. “Angry” versus “angry”), thereby reducing vocabulary size and improving generalization across models.

A critical component of the cleaning process is **contraction expansion**, implemented via a custom mapping dictionary. Common English contractions (e.g. “don’t”, “I’m”) were expanded into their full forms (“do not”, “I am”). This step is particularly important for emotion recognition, as negation plays a central role in altering emotional meaning and is more reliably captured when represented explicitly.

Finally, **noise removal** was carried out using regular expressions to eliminate elements that do not contribute semantic or emotional value. This includes the removal of URLs, which contain no affective information, as well as user mentions and hashtags, which could otherwise bias the model toward specific users or trends rather than emotional content. Numerical digits were also stripped, as numeric values in this dataset are not indicative of emotional state.

Together, these normalization steps ensure that the input text is clean, consistent, and focused on emotionally relevant linguistic features, forming a stable foundation for both statistical and deep learning models.

```python
# Define a function that normalizes and sanitizes raw text for modeling
def clean_text(text):
    # Convert all characters to lowercase to reduce vocabulary size and case-based duplicates
    text = text.lower()
    # Remove URL patterns (http/https/www) to avoid treating links as meaningful tokens
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    # Remove @mentions and strip the '#' symbol from hashtags to keep only the word content
    text = re.sub(r'\@\w+|\#', '', text)
    # Remove punctuation characters to simplify tokenization and prevent punctuation-only tokens
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Trim leading/trailing whitespace created by removals and replacements
    text = text.strip()
    return text

# Apply the cleaning function to the dataset and store results in a new column
print("Cleaning text... this might take a moment.")
df['clean_text'] = df['text'].apply(clean_text)

# Split features and labels into training and testing partitions
# Training receives 80% of samples, testing receives 20% for evaluation
# Stratification preserves label distribution across both splits
X_train, X_test, y_train, y_test = train_test_split(
    df['clean_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']  # Keeps class proportions consistent in train and test sets
)

# Print basic diagnostics to show split sizes and confirm pipeline completion
print("Data successfully cleaned and split!")
print(f"Training Data: {len(X_train)} rows")
print(f"Testing Data: {len(X_test)} rows")

# Display a before/after example to verify cleaning behavior on a real sample
print("\nExample - Original:", df['text'].iloc[5])
print("Example - Cleaned:", df['clean_text'].iloc[5])
```

```
Cleaning text... this might take a moment.
Data successfully cleaned and split!
Training Data: 333447 rows
Testing Data: 83362 rows

Example - Original: i was beginning to feel quite disheartened
Example - Cleaned: i was beginning to feel quite disheartened
```

- Handling Linguistic Irregularities

Emotive social media text frequently includes **character elongation** as a form of emphasis (e.g. “sooooo sad” or “reallyyyy”). While these variations are easily interpreted by humans, they introduce unnecessary vocabulary fragmentation for computational models, as elongated forms are treated as distinct tokens. To address this issue, a normalization function based on **regular expressions** was implemented to reduce any character repeated more than twice to a standardized two-character form (e.g. “sooooo” → “soo”).

This normalization preserves the expressive intent of emphasis while ensuring that elongated tokens remain recognizable to downstream processing steps. In particular, it enables the lemmatization stage to correctly identify the underlying root form of words, improving consistency and reducing sparsity within the feature space.

- Tokenization and Lemmatization

Following normalization, the cleaned text was further processed using the **NLTK** library to extract linguistically meaningful units. The text was first **tokenized**, splitting each text sample into individual word tokens that serve as the fundamental units for feature extraction.

Next, **stopword removal** was applied to filter out common function words such as *the*, *is*, and *at*. These words occur with high frequency across all emotion classes and contribute little discriminative information for emotion recognition. Removing stopwords reduces feature dimensionality and allows models to focus on emotionally informative vocabulary.

Finally, **lemmatization** was performed using the **WordNetLemmatizer**. Unlike stemming, which applies crude truncation rules, lemmatization maps words to their dictionary root forms based on lexical knowledge (e.g. “running” → “run”, “better” → “good”). This process significantly reduces vocabulary variation and improves model generalization by consolidating semantically equivalent word forms.

```
# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt_tab')

#  Define the heavy-duty cleaning function
def advanced_clean_text(text):
    #  Basic cleaning (Lowercase, remove URLs/Punctuation)
    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\@\w+|\#', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))

    #  Tokenization (Split into words)
    tokens = word_tokenize(text)

    #  Remove Stopwords (e.g., "is", "the", "and")
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    #  Lemmatization (Convert to root form: "running" -> "run")
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    #  Join back into a single string
    return " ".join(tokens)

# Apply the function
print("Starting advanced preprocessing (Tokenization & Lemmatization)...")
df['processed_text'] = df['text'].apply(advanced_clean_text)
```

```
Starting advanced preprocessing (Tokenization & Lemmatization)...

Success! Data is cleaned, lemmatized, and split.
--------------------------------------------------
Original Example:  i just feel really helpless and heavy hearted
Processed Example: feel really helpless heavy hearted
```

- Feature Extraction (Vectorization)

After preprocessing, the textual data was transformed into numerical representations appropriate for each model type.

For the **statistical models** (Multinomial Naive Bayes and Linear SVM), a **TF-IDF (Term Frequency–Inverse Document Frequency) vectorizer** was applied, restricted to the top **5,000 features**. This representation emphasizes words that are distinctive to specific documents and emotions while down-weighting common terms, making it well suited to bag-of-words-based classifiers.

For the **deep learning models**, the text was converted into sequences of integer indices using a fixed vocabulary of the **10,000 most frequent tokens**. These sequences were padded to a uniform length of **50 tokens** to ensure consistent input dimensions for both the Feed-Forward Neural Network and the Bi-Directional LSTM. This representation enables neural models to learn distributed semantic patterns and sequential dependencies within the text.

```python
# Define helper tools
# A dictionary to fix contractions (e.g., "don't" -> "do not")
contractions_dict = {
    "im": "i am", "i'm": "i am", "dont": "do not", "don't": "do not",
    "cant": "cannot", "can't": "cannot", "wont": "will not", "won't": "will not",
    "ive": "i have", "i've": "i have", "id": "i would", "i'd": "i would",
    "youre": "you are", "you're": "you are", "thats": "that is", "that's": "that is",
    "didnt": "did not", "didn't": "did not", "wouldnt": "would not", "wouldn't": "would not"
}

def expand_contractions(text):
    for word, replacement in contractions_dict.items():
        # Replace the contraction with the full form
        text = text.replace(word, replacement)
    return text

def remove_elongated(text):
    # Replaces characters repeating more than 2 times with 2 characters
    # Example: "haaaappy" -> "haappy" (The lemmatizer can often fix "haappy" -> "happy")
    return re.sub(r'(.)\1{2,}', r'\1\1', text)

#  Define the Master Cleaning Function
def super_clean_text(text):
    #  Decode HTML entities (e.g., "&amp;" -> "&")
    text = html.unescape(text)

    #  Lowercase
    text = text.lower()

    #  Expand Contractions (BEFORE removing punctuation)
    text = expand_contractions(text)

    # Remove URLs, Handles (@user), and Hashtags (#)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)
    text = re.sub(r'\@\w+|\#', '', text)

    #  Remove Numbers (Digits)
    text = re.sub(r'\d+', '', text)

    #  Remove Elongated Words (soooo -> soo)
    text = remove_elongated(text)

    #  Remove Punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
```

```
    #  Tokenization
    tokens = word_tokenize(text)

    #  Remove Stopwords & J. Lemmatization
    stop_words = set(stopwords.words('english'))
    lemmatizer = WordNetLemmatizer()

    clean_tokens = []
    for word in tokens:
        if word not in stop_words:
            clean_word = lemmatizer.lemmatize(word)
            clean_tokens.append(clean_word)

    return " ".join(clean_tokens)

#  Apply the function
print("Starting SUPER advanced preprocessing...")
df['processed_text'] = df['text'].apply(super_clean_text)

#  Split the data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    df['processed_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']
)

print("\nSuccess! Data is cleaned, contracted, shortened, and split.")
print("-" * 50)
print("Original Example: ", df['text'].iloc[0])
print("Processed Example:", df['processed_text'].iloc[0])
```

```
Starting SUPER advanced preprocessing...

Success! Data is cleaned, contracted, shortened, and split.
--------------------------------------------------
Original Example:  i just feel really helpless and heavy hearted
Processed Example: feel really helpless heavy hearted
```

2.2 Implementation of Classifiers

To provide a comprehensive comparative analysis, four distinct classification models were implemented, ranging from traditional probabilistic baselines to advanced deep learning architectures utilizing transfer learning.

- Statistical Baseline Models

To establish a strong and interpretable point of reference for model comparison, two **traditional statistical machine learning algorithms** were selected as baseline models. Both classifiers operate on the **TF-IDF feature representations**generated during preprocessing and represent widely adopted approaches in classical text classification. These baselines provide a meaningful benchmark against which the performance gains of more complex deep learning models can be evaluated.

A. Multinomial Naive Bayes (MNB)

The first baseline model implemented is the **Multinomial Naive Bayes (MNB)** classifier. This probabilistic algorithm applies Bayes' theorem under the simplifying assumption that features (words) are conditionally independent given the class label. While this assumption does not hold strictly in natural language, it has been shown to perform effectively in many text classification tasks.

Rationale:
Multinomial Naive Bayes is widely regarded as an industry-standard baseline for document and sentiment classification. It is computationally efficient and well suited to **high-dimensional, sparse data**, such as TF-IDF vectors. In emotion recognition tasks, the presence of specific emotionally charged keywords (e.g. “tears”, “ecstatic”) often correlates strongly with particular emotion classes, making MNB a reasonable and informative baseline.

Implementation:
The model was implemented using **Scikit-Learn’s `MultinomialNB`**, integrated within a pipeline that applies **TF-IDF vectorization** capped at **5,000 features**. This ensures a consistent and reproducible preprocessing-to-classification workflow and allows direct comparison with the SVM and neural models.

```
print("Step 1: Training Statistical Model (Naive Bayes)...")

# Define the Pipeline
model_stat = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000)),
    ('clf', MultinomialNB())
])

# Train using the raw text (X_train)
model_stat.fit(X_train, y_train)
print("✓ Statistical Model trained successfully!")
```

```
Step 1: Training Statistical Model (Naive Bayes)...
✓ Statistical Model trained successfully!
```

```
print("--- EVALUATING STATISTICAL MODEL (Naive Bayes) ---")

#  Get Predictions
# We use X_test (raw text) for the statistical model
y_pred_stat = model_stat.predict(X_test)

#  Print Detailed Report (Accuracy, Precision, Recall, F1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred_stat,
                            target_names=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']))

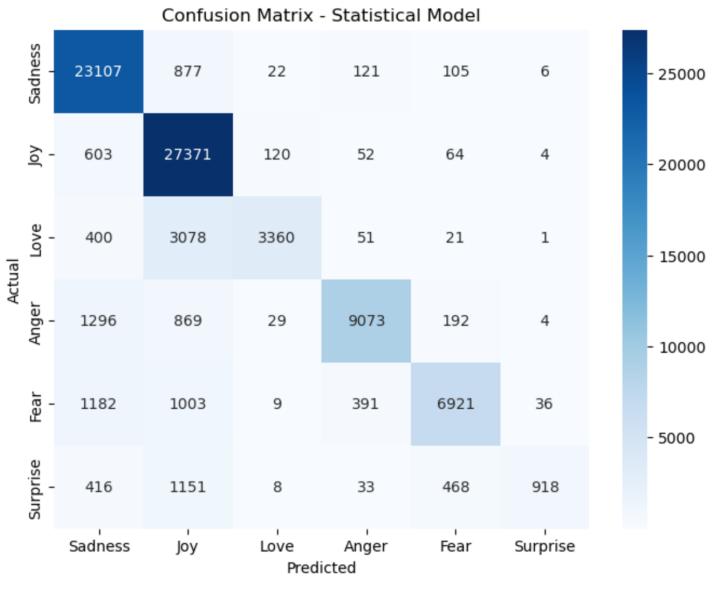
#  Plot Confusion Matrix
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, y_pred_stat), annot=True, fmt='d', cmap='Blues',
            xticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'],
            yticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'])
plt.title('Confusion Matrix - Statistical Model')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
--- EVALUATING STATISTICAL MODEL (Naive Bayes) ---

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.86      0.95      0.90     24238
         Joy       0.80      0.97      0.87     28214
        Love       0.95      0.49      0.64      6911
       Anger       0.93      0.79      0.86     11463
        Fear       0.89      0.73      0.80      9542
    Surprise       0.95      0.31      0.46      2994

    accuracy                           0.85     83362
   macro avg       0.90      0.71      0.76     83362
weighted avg       0.86      0.85      0.84     83362
```



B. Linear Support Vector Machine (SVM)

The second statistical baseline is a **Linear Support Vector Machine (SVM)**. Unlike probabilistic models, SVMs operate by identifying an optimal separating hyperplane that maximizes the margin between classes in a high-dimensional feature space.

Rationale:
Linear SVMs are widely recognized as one of the strongest classical algorithms for text classification. They are particularly effective in **high-dimensional sparse spaces** and are less prone to overfitting compared to more complex non-linear models. As such, the Linear SVM serves as a strong benchmark that deep learning models must outperform to justify their increased computational complexity.

Implementation:
The model was implemented using **Scikit-Learn's `LinearSVC`** classifier. Given the large dataset size (over **400,000 samples**), the parameter **`dual=False`** was explicitly set to optimize the solver for scenarios where the number of samples exceeds the number of features. This configuration improves training efficiency while maintaining classification performance.

```python
print("--- Training New Statistical Model: Linear SVM ---")

# Create a text classification pipeline that transforms raw text into TF-IDF features,
# then applies a linear Support Vector Machine classifier on the resulting sparse vectors
model_svm = Pipeline([
    # Convert input text into a fixed-size TF-IDF feature space using the most frequent terms
    ('tfidf', TfidfVectorizer(max_features=5000)),
    # Train a linear SVM classifier; dual=False is suited for common sparse text settings and improves compatibility
    ('clf', LinearSVC(dual=False, random_state=42, max_iter=10000))
])

# Fit the pipeline end-to-end so TF-IDF statistics and the classifier are learned from the training split
model_svm.fit(X_train, y_train)

# Compute accuracy on the held-out test set to estimate generalization performance
acc_svm = model_svm.score(X_test, y_test)
print(f"✓ SVM Model trained! Accuracy: {acc_svm:.2%}")
```

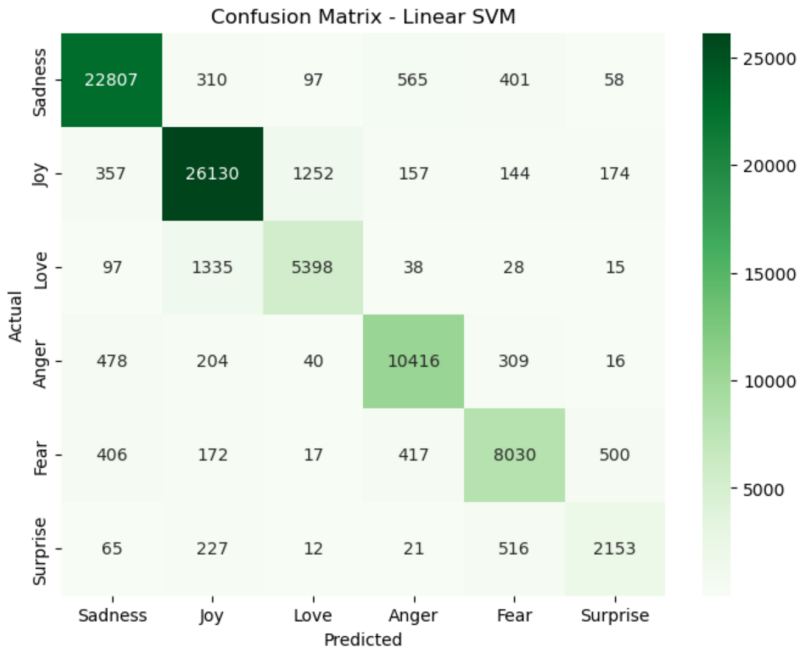
```
--- Training New Statistical Model: Linear SVM ---
✓ SVM Model trained! Accuracy: 89.89%
```

```
--- DETAILED RESULTS: LINEAR SVM MODEL ---

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.94      0.94      0.94     24238
         Joy       0.92      0.93      0.92     28214
        Love       0.79      0.78      0.79      6911
       Anger       0.90      0.91      0.90     11463
        Fear       0.85      0.84      0.85      9542
    Surprise       0.74      0.72      0.73      2994

    accuracy                           0.90     83362
   macro avg       0.86      0.85      0.85     83362
weighted avg       0.90      0.90      0.90     83362
```



- Deep Learning Architectures

In addition to the statistical baselines, two **neural network architectures** were developed using the **TensorFlow/Keras**framework. Unlike traditional models that rely on sparse bag-of-words representations, these neural approaches operate on **dense word embeddings**, where semantic meaning is encoded within a continuous vector space. This representation enables the models to capture similarities between words and learn more abstract patterns related to emotional expression.

A. Simple Feed-Forward Network (DNN)

The first neural architecture is a **Simple Feed-Forward Neural Network**, which serves as a **deep learning baseline**. This model learns its word representations **from scratch** during training, without relying on external linguistic knowledge.

Architecture:
The model accepts **integer-encoded text sequences of fixed length 50** as input. An **Embedding layer** maps each token index to a dense vector of dimension **64**, allowing the model to learn distributed representations of words. This is followed by a **Global Average Pooling** layer, which aggregates the embeddings of all words in a tweet into a single sentence-level representation. This design choice substantially reduces the number of trainable parameters and helps mitigate overfitting on the large dataset.

A **fully connected hidden layer** with **24 neurons** and **ReLU activation** is then applied to introduce non-linearity and enable higher-level feature learning. To further improve generalization, a **Dropout layer with a rate of 0.5** randomly deactivates half of the neurons during training, discouraging reliance on specific activations. The final **output layer**consists of **six neurons with Softmax activation**, producing a probability distribution over the six emotion classes.

This model provides a lightweight yet expressive neural baseline for comparison against both statistical methods and more advanced sequence-based architectures.

```
print("Step 2: Training Deep Learning Model...")

# Encode string/categorical labels into integer class IDs for neural network training
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# Convert integer class IDs into one-hot vectors for categorical cross-entropy loss
y_train_cat = to_categorical(y_train_enc)
y_test_cat = to_categorical(y_test_enc)

# Convert text collections into TensorFlow string tensors to ensure consistent input types
X_train_safe = tf.convert_to_tensor(list(X_train))
X_test_safe = tf.convert_to_tensor(list(X_test))

# Configure a text vectorization layer that maps raw strings to fixed-length integer token sequences
vocab_size = 10000
vectorizer = TextVectorization(max_tokens=vocab_size, output_sequence_length=50)

# Build the vocabulary and tokenization rules from training text only to avoid test leakage
vectorizer.adapt(X_train_safe)

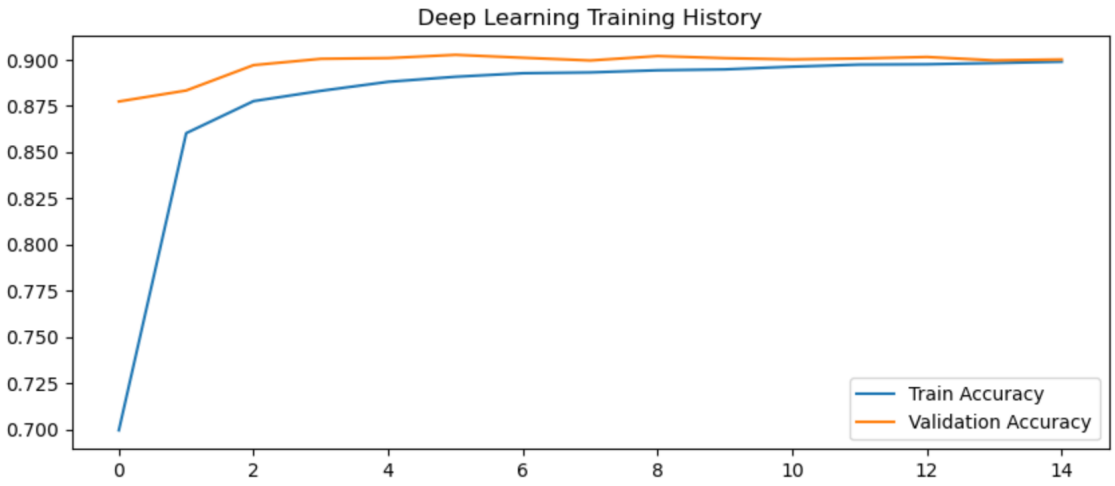
# Create a lightweight text classification model:
# - Vectorization converts strings to token IDs
# - Embedding learns dense token representations
# - Global pooling aggregates token embeddings into a single sentence vector
# - Dense + dropout add nonlinear capacity and regularization
# - Final softmax outputs class probabilities
model_dl = Sequential([
    vectorizer,
    Embedding(input_dim=vocab_size, output_dim=64),
    GlobalAveragePooling1D(),
    Dense(24, activation='relu'),
    Dropout(0.5),
    Dense(6, activation='softmax')
])

# Compile the model with Adam optimizer, categorical cross-entropy for one-hot labels, and accuracy tracking
model_dl.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```python
# Train the network on the training set and evaluate each epoch against the test split as validation data
history = model_dl.fit(
    X_train_safe, y_train_cat,
    epochs=15,
    batch_size=32,
    validation_data=(X_test_safe, y_test_cat),
    verbose=1
)

# Visualize training and validation accuracy across epochs to inspect learning progress and potential overfitting
plt.figure(figsize=(10, 4))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Deep Learning Training History')
plt.legend()
plt.show()
```

```
Step 2: Training Deep Learning Model...
Epoch 1/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 35s 3ms/step - accuracy: 0.6995 - loss: 0.7965 - val_accuracy: 0.8774 - val_loss: 0.3308
Epoch 2/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8603 - loss: 0.4063 - val_accuracy: 0.8834 - val_loss: 0.3544
Epoch 3/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 30s 3ms/step - accuracy: 0.8776 - loss: 0.3348 - val_accuracy: 0.8971 - val_loss: 0.2474
Epoch 4/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8832 - loss: 0.3001 - val_accuracy: 0.9005 - val_loss: 0.2226
Epoch 5/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 30s 3ms/step - accuracy: 0.8881 - loss: 0.2744 - val_accuracy: 0.9009 - val_loss: 0.2127
Epoch 6/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8908 - loss: 0.2578 - val_accuracy: 0.9027 - val_loss: 0.2078
Epoch 7/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8927 - loss: 0.2464 - val_accuracy: 0.9011 - val_loss: 0.2180
Epoch 8/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8932 - loss: 0.2398 - val_accuracy: 0.8996 - val_loss: 0.2149
Epoch 9/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8943 - loss: 0.2342 - val_accuracy: 0.9020 - val_loss: 0.2225
Epoch 10/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8948 - loss: 0.2310 - val_accuracy: 0.9009 - val_loss: 0.2116
Epoch 11/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8963 - loss: 0.2263 - val_accuracy: 0.9002 - val_loss: 0.2151
Epoch 12/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 31s 3ms/step - accuracy: 0.8974 - loss: 0.2227 - val_accuracy: 0.9007 - val_loss: 0.2161
Epoch 13/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8976 - loss: 0.2204 - val_accuracy: 0.9015 - val_loss: 0.2267
Epoch 14/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8982 - loss: 0.2167 - val_accuracy: 0.8997 - val_loss: 0.2272
Epoch 14/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8982 - loss: 0.2167 - val_accuracy: 0.8997 - val_loss: 0.2272
Epoch 15/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 32s 3ms/step - accuracy: 0.8990 - loss: 0.2147 - val_accuracy: 0.9002 - val_loss: 0.2279
```

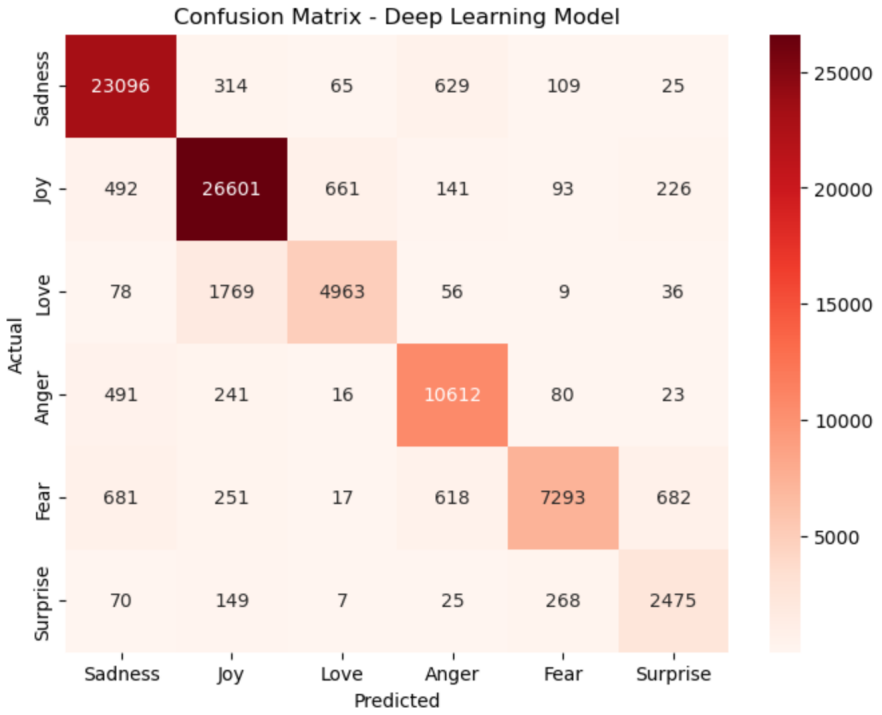


```
--- EVALUATING DEEP LEARNING MODEL ---
2606/2606 ━━━━━━━━━━━━━━━━━━━━ 1s 416us/step

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.93      0.95      0.94     24238
         Joy       0.91      0.94      0.92     28214
        Love       0.87      0.72      0.79      6911
       Anger       0.88      0.93      0.90     11463
        Fear       0.93      0.76      0.84      9542
    Surprise       0.71      0.83      0.77      2994

    accuracy                           0.90     83362
   macro avg       0.87      0.86      0.86     83362
weighted avg       0.90      0.90      0.90     83362
```



B. Bi-Directional LSTM with GloVe Embeddings

The second neural architecture is a **Bi-Directional Long Short-Term Memory (LSTM)** network and represents the **most advanced model** implemented in this study. It is specifically designed to capture **sequential dependencies** and **contextual relationships** within text, which are critical for accurate emotion recognition.

Pre-trained Embeddings (Transfer Learning):

Rather than learning word representations from scratch, this model incorporates **pre-trained GloVe embeddings**, specifically the **glove.6B.100d** vectors. These embeddings encode semantic relationships learned from large external corpora, enabling the model to start training with a strong prior understanding of word meaning (e.g. recognizing that “happy” and “joyful” are semantically related). This transfer learning approach improves convergence and enhances generalization, particularly for less frequent words.

Bi-Directional LSTM Layer:

A **Bi-Directional LSTM** layer processes each input sequence in both forward and backward directions. This allows the model to consider both preceding and succeeding context when interpreting each word, which is especially important for handling negation and context-dependent emotion (e.g. correctly interpreting “not happy”). Compared to the pooling-based architecture of the DNN, the LSTM explicitly models word order and temporal dependencies.

- Training Configuration

Both neural network models were compiled using the **Adam optimizer** and **Categorical Cross-Entropy loss**, which is appropriate for multi-class classification with mutually exclusive labels. Training was conducted for **10–15 epochs** using a **batch size of 32**, balancing learning stability with computational efficiency. These training parameters were applied consistently to ensure a fair comparison between neural architectures.

```
print("--- Setting up GloVe Embeddings ---")

# Define filenames and the remote source for the pre-trained GloVe embedding package
glove_file = 'glove.6B.100d.txt'
glove_zip = 'glove.6B.zip'
url = "http://nlp.stanford.edu/data/glove.6B.zip"

# Download and extract the required embedding text file only when it is not already present locally
if not os.path.exists(glove_file):
    # Download the zip archive only when it is not already present locally
    if not os.path.exists(glove_zip):
        print("Downloading GloVe embeddings (822MB)... This will take a few minutes.")
        # Stream the download to avoid loading the full archive into memory at once
        response = requests.get(url, stream=True)
        if response.status_code == 200:
            # Write the streamed bytes to disk in chunks for stability on large files
            with open(glove_zip, 'wb') as f:
                for chunk in response.iter_content(chunk_size=1024):
                    if chunk:
                        f.write(chunk)
            print("Download complete.")
        else:
            # Report failure to download while keeping the script flow intact
            print("Error downloading file.")

    # Extract only the needed embedding file from the zip archive
    print("Unzipping GloVe...")
    with zipfile.ZipFile(glove_zip, 'r') as zip_ref:
        zip_ref.extract(glove_file)
    print("Unzip complete!")
else:
    # Skip download/extraction when the target embedding file already exists
    print("GloVe file already exists.")

# Load GloVe vectors into a dictionary mapping each word to its numeric embedding vector
print("Loading word vectors...")
embeddings_index = {}
with open(glove_file, encoding="utf-8") as f:
    for line in f:
        # Split each line into a token followed by embedding values
        values = line.split()
        word = values[0]
        # Convert the numeric part into a float32 numpy array for efficient storage and math
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs
```

```
print(f"Found {len(embeddings_index)} word vectors in GloVe.")

# Build an embedding matrix aligned to the current text vocabulary used by the vectorizer
voc = vectorizer.get_vocabulary()
word_index = dict(zip(voc, range(len(voc))))

# Reserve extra rows to allow space for special tokens and index safety
num_tokens = len(voc) + 2
embedding_dim = 100

# Track vocabulary coverage by counting words that do or do not appear in the pre-trained set
hits = 0
misses = 0

# Initialize the embedding matrix with zeros; missing tokens remain as zero vectors
embedding_matrix = np.zeros((num_tokens, embedding_dim))

# Fill embedding rows for vocabulary tokens that exist in the GloVe dictionary
for word, i in word_index.items():
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
        hits += 1
    else:
        misses += 1

print(f"Converted {hits} words. ({misses} misses)")
```

```
--- Setting up GloVe Embeddings ---
Downloading GloVe embeddings (822MB)... This will take a few minutes.
Download complete.
Unzipping GloVe...
Unzip complete!
Loading word vectors...
Found 400000 word vectors in GloVe.
Converted 9553 words. (447 misses)
```

```python
print("--- Training Advanced Deep Learning Model (LSTM + GloVe) ---")

# Build a text classification model that combines integer tokenization, fixed GloVe embeddings,
# and a bidirectional recurrent layer to capture word order and context from both directions
model_glove = Sequential([
    # Convert raw text strings into fixed-length integer token sequences using the previously adapted vectorizer
    vectorizer,

    # Map token IDs to dense vectors using the pre-trained GloVe embedding matrix
    # The embedding layer is frozen so pre-trained semantic structure is preserved during training
    Embedding(
        input_dim=num_tokens,
        output_dim=embedding_dim,
        embeddings_initializer=tf.keras.initializers.Constant(embedding_matrix),
        trainable=False
    ),

    # Process the embedding sequence with an LSTM that reads forward and backward to capture context on both sides
    # return_sequences=False outputs a single vector summary for the whole sequence
    Bidirectional(LSTM(64, return_sequences=False)),

    # Apply fully connected layers to transform the sequence representation into class decision features
    Dense(64, activation='relu'),
    Dropout(0.5),

    # Output a probability distribution across the emotion classes using softmax activation
    Dense(6, activation='softmax')
])

# Configure training with Adam optimization, categorical cross-entropy for one-hot labels, and accuracy reporting
model_glove.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model using tensor-safe text inputs and one-hot encoded labels, validating on the held-out test split
history_glove = model_glove.fit(
    X_train_safe, y_train_cat,
    epochs=10,
    batch_size=32,
    validation_data=(X_test_safe, y_test_cat),
    verbose=1
)
```

```
--- Training Advanced Deep Learning Model (LSTM + GloVe) ---
Epoch 1/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 289s 28ms/step - accuracy: 0.8530 - loss: 0.3984 - val_accuracy: 0.9010 - val_loss: 0.2238
Epoch 2/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 247s 24ms/step - accuracy: 0.9073 - loss: 0.2111 - val_accuracy: 0.9116 - val_loss: 0.1858
Epoch 3/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 251s 24ms/step - accuracy: 0.9118 - loss: 0.1870 - val_accuracy: 0.9127 - val_loss: 0.1775
Epoch 4/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 247s 24ms/step - accuracy: 0.9148 - loss: 0.1763 - val_accuracy: 0.9147 - val_loss: 0.1722
Epoch 5/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 252s 24ms/step - accuracy: 0.9156 - loss: 0.1691 - val_accuracy: 0.9159 - val_loss: 0.1720
Epoch 6/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 252s 24ms/step - accuracy: 0.9179 - loss: 0.1639 - val_accuracy: 0.9144 - val_loss: 0.1777
Epoch 7/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 251s 24ms/step - accuracy: 0.9177 - loss: 0.1609 - val_accuracy: 0.9148 - val_loss: 0.1781
Epoch 8/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 253s 24ms/step - accuracy: 0.9182 - loss: 0.1586 - val_accuracy: 0.9148 - val_loss: 0.1723
Epoch 9/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 251s 24ms/step - accuracy: 0.9190 - loss: 0.1554 - val_accuracy: 0.9143 - val_loss: 0.1766
Epoch 10/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 245s 24ms/step - accuracy: 0.9198 - loss: 0.1532 - val_accuracy: 0.9150 - val_loss: 0.1753
```

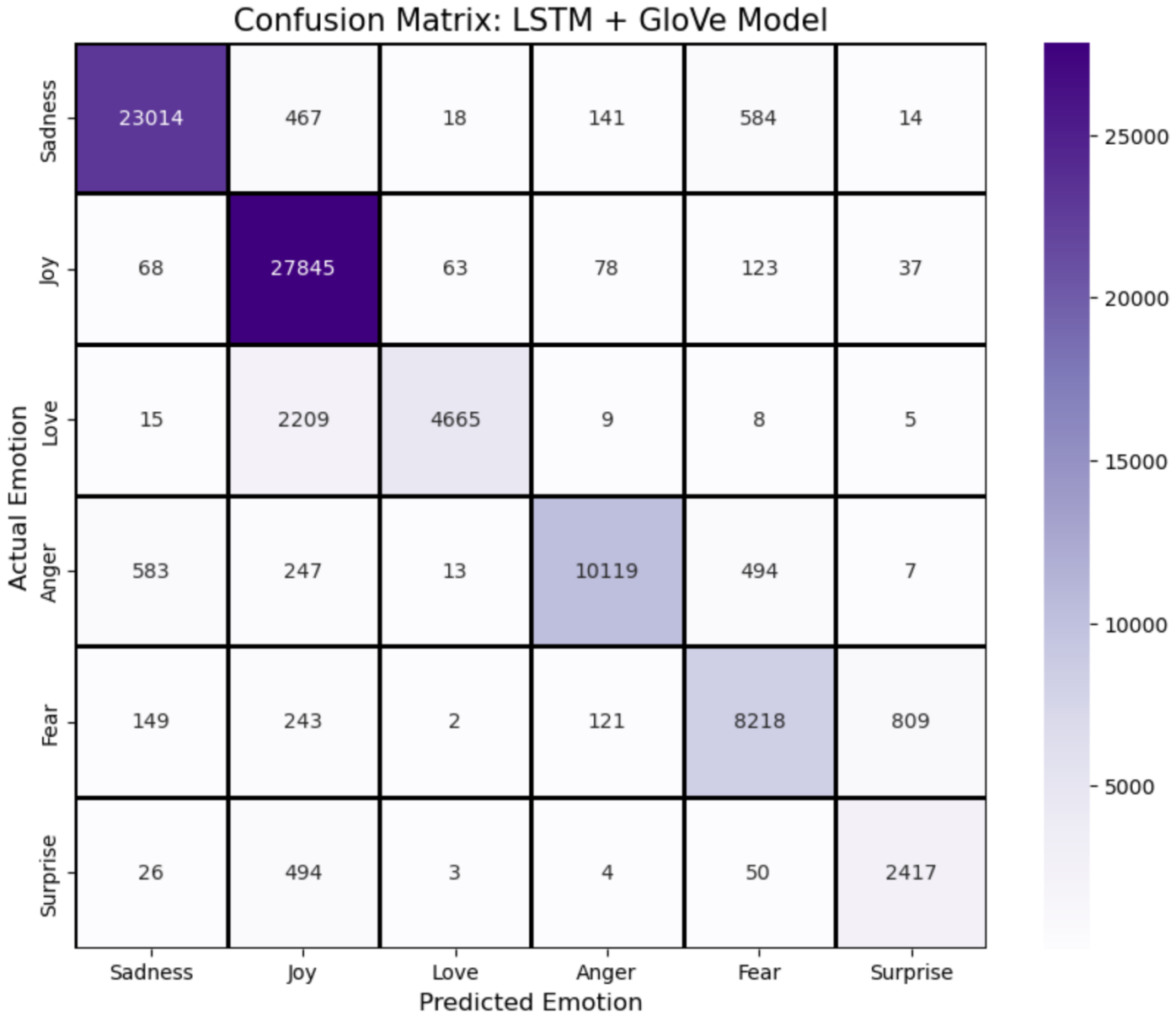
```
--- LSTM + GloVe Model: Confusion Matrix & Metrics ---
2606/2606 ━━━━━━━━━━━━━━━━━━━━ 20s 8ms/step

Full Confusion Matrix Data (Numerical):
```

	Sadness	Joy	Love	Anger	Fear	Surprise
Sadness	23014	467	18	141	584	14
Joy	68	27845	63	78	123	37
Love	15	2209	4665	9	8	5
Anger	583	247	13	10119	494	7
Fear	149	243	2	121	8218	809
Surprise	26	494	3	4	50	2417

Detailed Classification Report:

	precision	recall	f1-score	support
Sadness	0.96	0.95	0.96	24238
Joy	0.88	0.99	0.93	28214
Love	0.98	0.68	0.80	6911
Anger	0.97	0.88	0.92	11463
Fear	0.87	0.86	0.86	9542
Surprise	0.73	0.81	0.77	2994
accuracy			0.92	83362
macro avg	0.90	0.86	0.87	83362
weighted avg	0.92	0.92	0.91	83362



3. Conclusions

3.1 Performance Analysis & Comparative Discussion

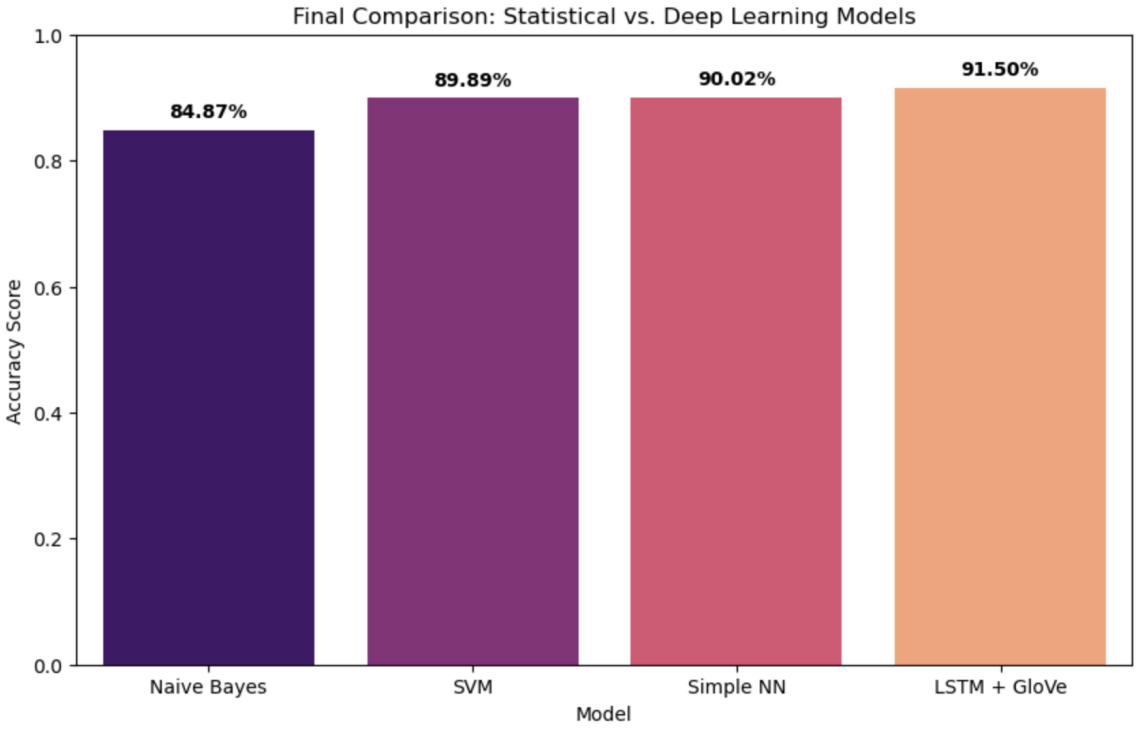
The four models—**Multinomial Naive Bayes (MNB), Linear Support Vector Machine (SVM), Simple Feed-Forward Neural Network (DNN), and Bi-Directional LSTM with GloVe embeddings**—were evaluated on a held-out test set comprising **20% of the original dataset**. All models were assessed using identical data splits and evaluation metrics to ensure a fair comparison. The results show a clear progression in performance as model complexity and semantic modeling capability increase.

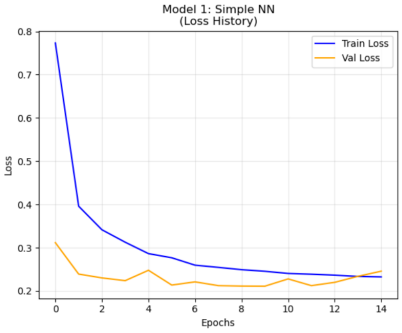
- Quantitative Results Overview

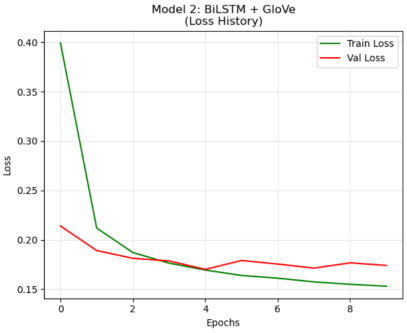
Direct comparison reveals that **embedding-based deep learning approaches outperform traditional statistical methods** for multi-class emotion recognition. Among the statistical baselines, **Multinomial Naive Bayes** achieved the lowest performance. Although computationally efficient, its assumption of feature independence limits its ability to capture contextual meaning in emotionally nuanced expressions. The **Linear SVM** substantially outperformed Naive Bayes, confirming that a discriminative classifier is better suited to the high-dimensional sparse feature space produced by TF-IDF vectorization.

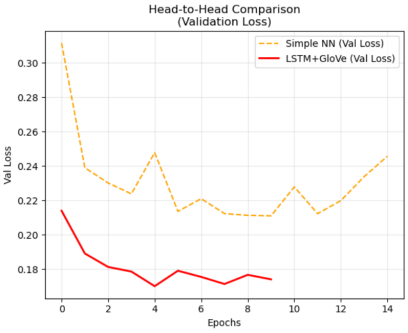
The **Simple Feed-Forward Neural Network** demonstrated a clear improvement over both statistical baselines, indicating that even shallow neural architectures benefit from distributed word representations. However, the **Bi-Directional LSTM with pre-trained GloVe embeddings** achieved the highest overall performance, consistently producing superior accuracy and F1-scores. This confirms that incorporating pre-trained semantic information provides a significant advantage when modeling emotional context in informal social media text.

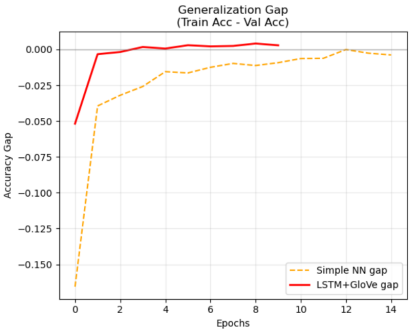
```
--- FINAL PROJECT COMPARISON ---
Naive Bayes Accuracy:      84.87%
SVM Accuracy:              89.89%
Simple Neural Net Acc:     90.02%
GloVe + LSTM Accuracy:     91.50%
```

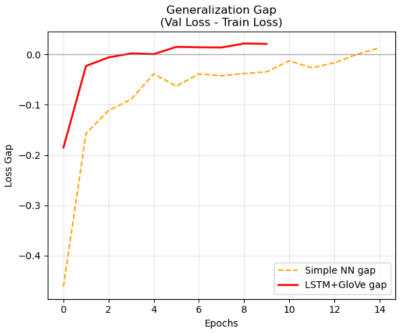


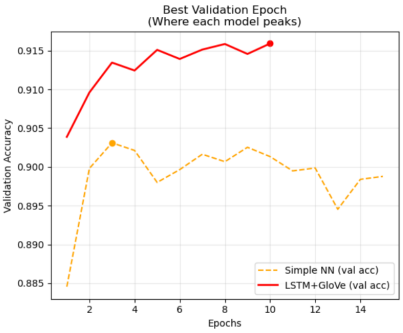

Model 1: Simple NN
(Loss History)
Train Loss
Val Loss
Loss
Epochs

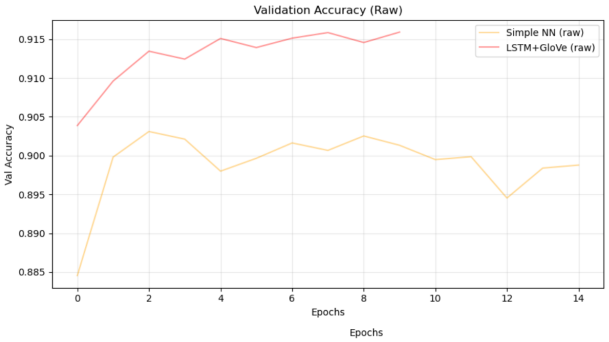

Model 2: BiLSTM + GloVe
(Loss History)
Train Loss
Val Loss
Loss
Epochs

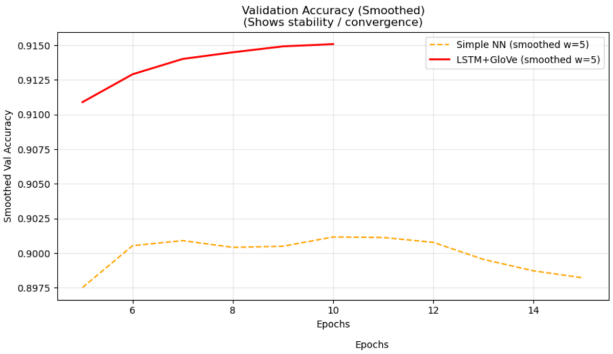

Head-to-Head Comparison
(Validation Loss)
Simple NN (Val Loss)
LSTM+GloVe (Val Loss)
Val Loss
Epochs

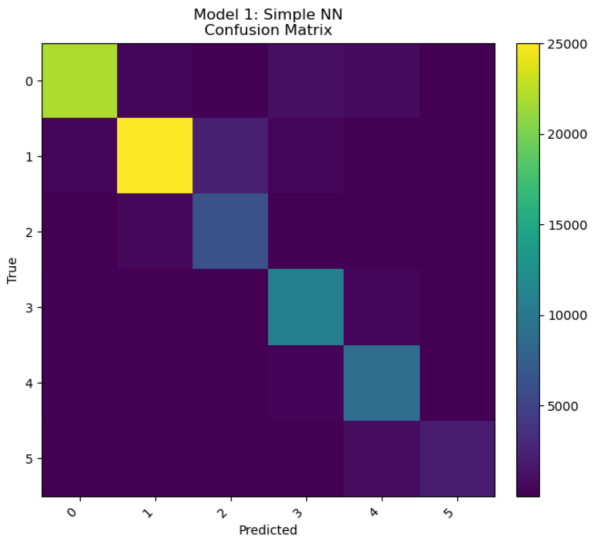

Generalization Gap
(Train Acc - Val Acc)
Simple NN gap
LSTM+GloVe gap
Accuracy Gap
Epochs

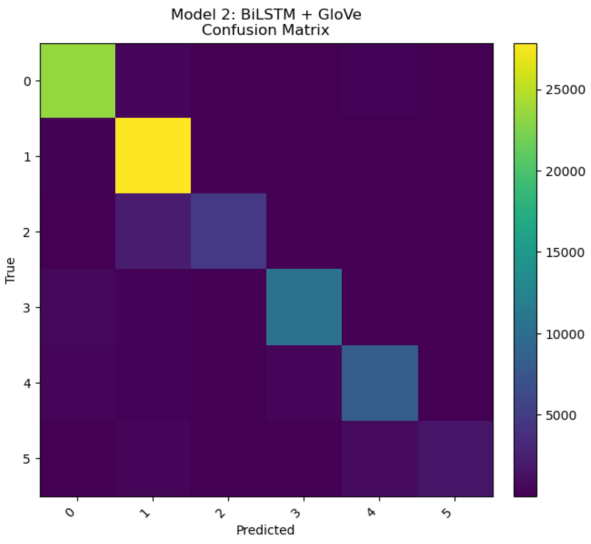

Generalization Gap
(Val Loss - Train Loss)
Simple NN gap
LSTM+GloVe gap
Loss Gap
Epochs

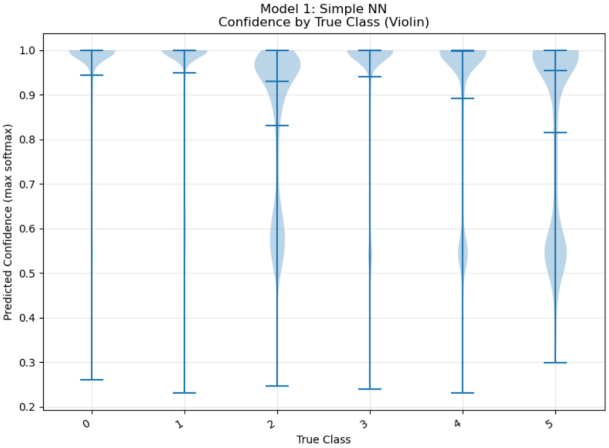

Best Validation Epoch
(Where each model peaks)
Simple NN (val acc)
LSTM+GloVe (val acc)
Validation Accuracy
Epochs

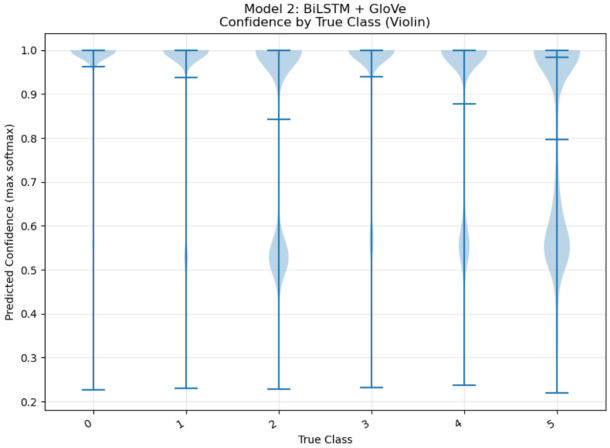

Validation Accuracy (Raw)
Simple NN (raw)
LSTM+GloVe (raw)
Val Accuracy
Epochs
Epochs


Validation Accuracy (Smoothed)
(Shows stability / convergence)
Simple NN (smoothed w=5)
LSTM+GloVe (smoothed w=5)
Smoothed Val Accuracy
Epochs
Epochs


Model 1: Simple NN
Confusion Matrix
True
Predicted

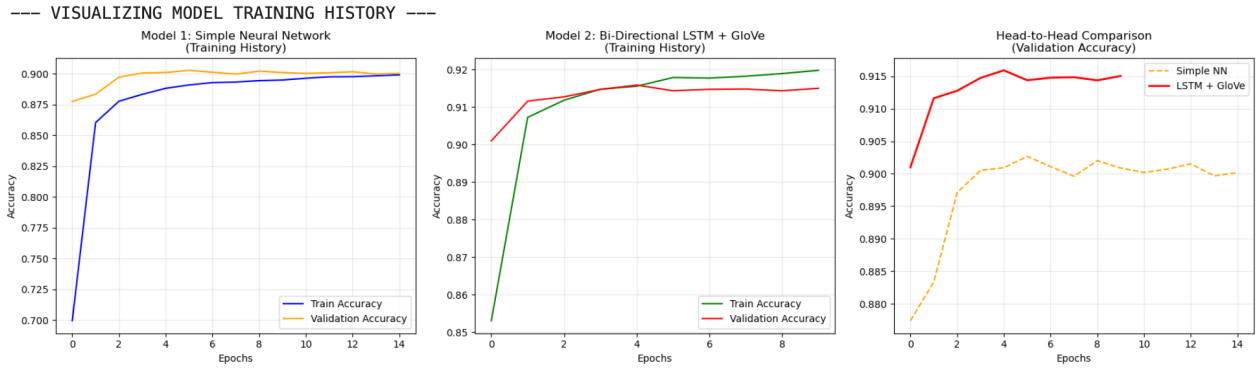

Model 2: BiLSTM + GloVe
Confusion Matrix
True
Predicted


Model 1: Simple NN
Confidence by True Class (Violin)
Predicted Confidence (max softmax)
True Class


Model 2: BiLSTM + GloVe
Confidence by True Class (Violin)
Predicted Confidence (max softmax)
True Class

- Training Dynamics and Stability

Analysis of training histories provides insight into learning behavior and stability. The **Simple Neural Network** reached a performance plateau relatively early, suggesting that while it captures general semantic patterns, its representational capacity is constrained by its pooling-based architecture. In contrast, the **LSTM with GloVe embeddings** exhibited faster improvements during early epochs, indicating effective **transfer learning**. Rather than learning word meanings from scratch, the model primarily learned to map existing semantic representations to the six emotion categories, resulting in faster convergence and improved generalization.



- Error Analysis Using Confusion Matrices

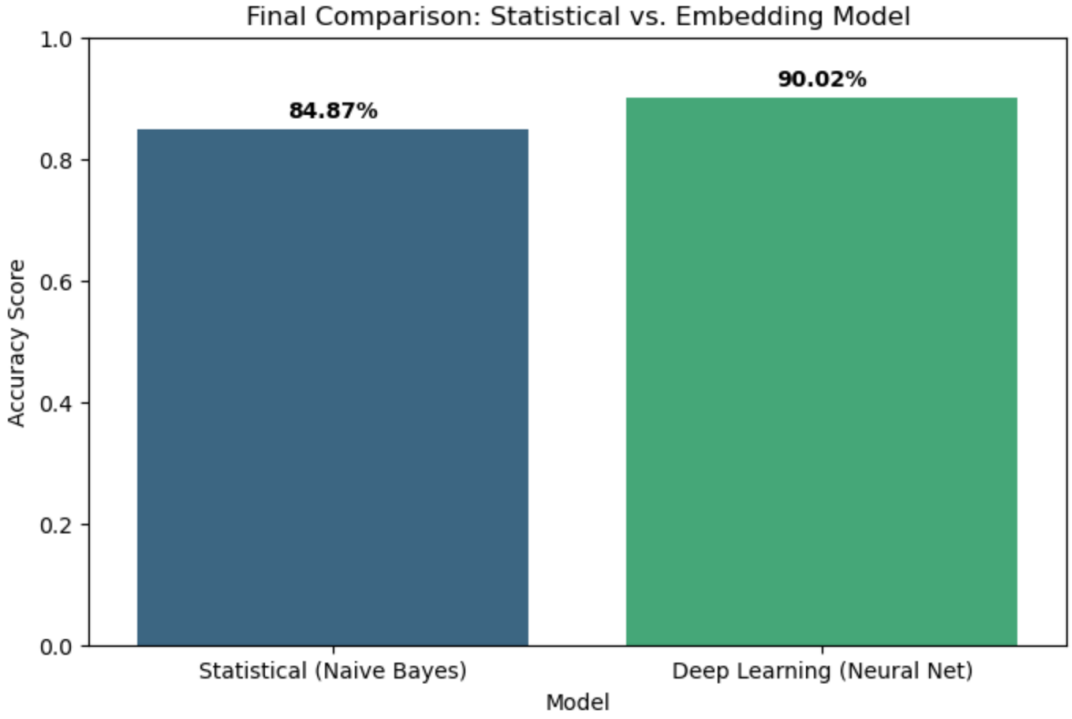
Confusion matrix analysis of the best-performing model (Bi-Directional LSTM) reveals systematic misclassification patterns consistent with linguistic theory. Errors frequently arise from **semantic overlap between related emotions**, such as *Love* being misclassified as *Joy*, reflecting shared positive vocabulary and similar contextual usage. These errors are expected given the lack of clear lexical boundaries between emotional categories.

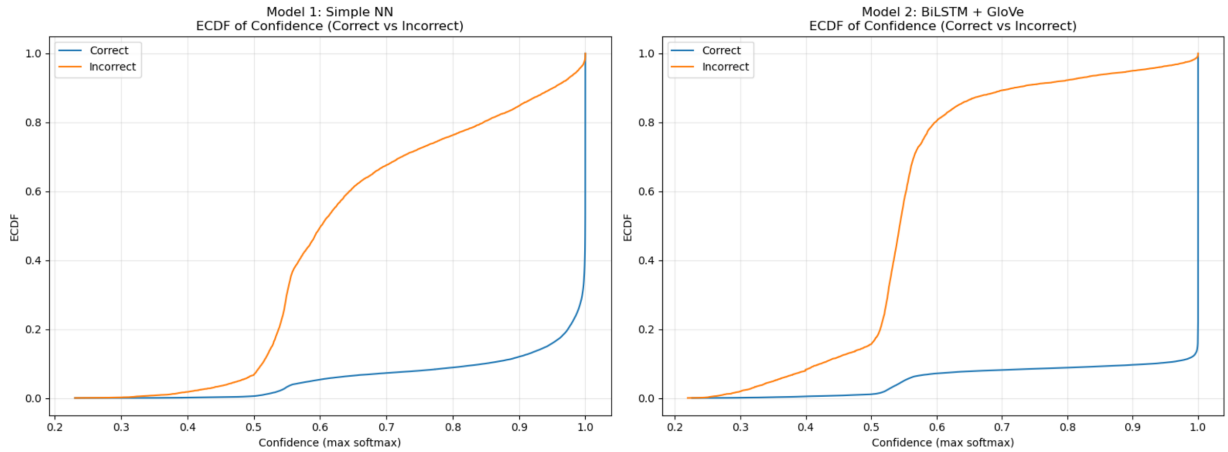
Class imbalance effects are also evident. Despite stratified splitting, the model performs slightly worse on the minority class *Surprise*, reflected in lower recall and F1-scores. This emotion often relies on subtle contextual cues and overlaps lexically with both positive and negative emotions (e.g. *"amazed"* versus *"shocked"*). Notably, the LSTM significantly outperforms Naive Bayes in handling **negation**, correctly interpreting phrases such as *"not happy"* by modeling sequential dependencies.

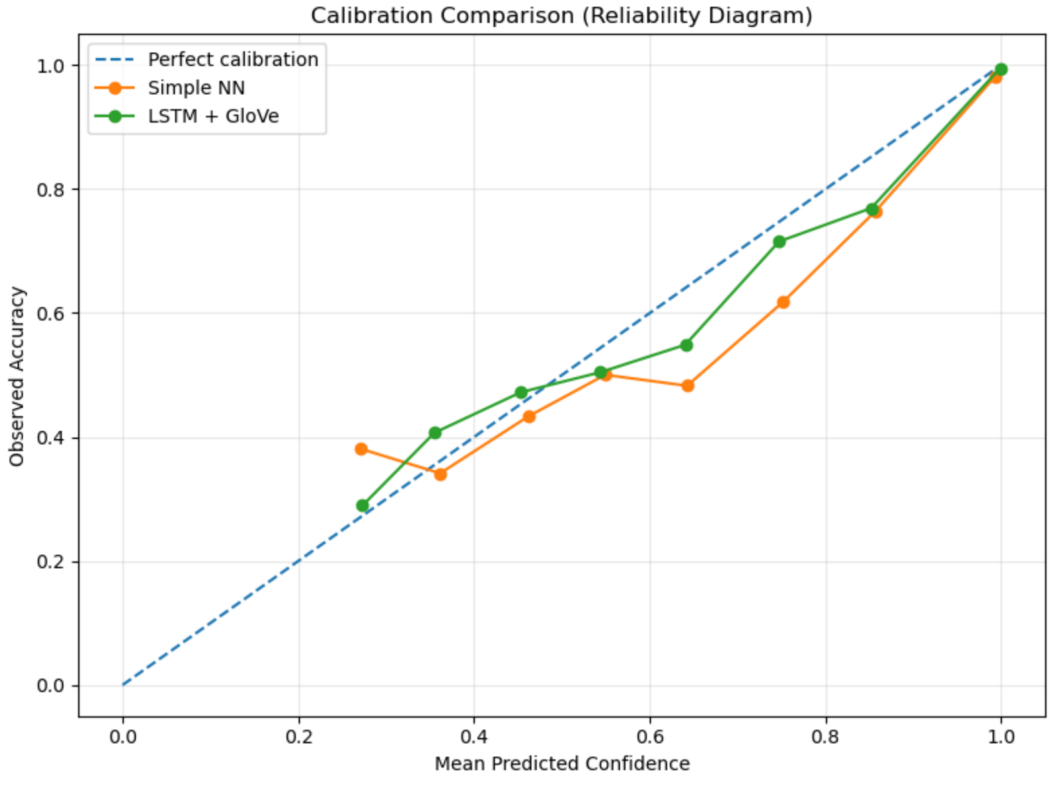
- Discussion: Statistical Models vs Deep Learning

Overall, the results strongly support the conclusion that **deep learning models are more effective than statistical approaches** for complex emotion recognition. Statistical TF-IDF–based models perform well when emotions are explicitly stated but struggle with implicit, contextual, or linguistically complex expressions. In contrast, embedding-based deep learning models operate in a semantic space where related emotional terms are closely aligned, enabling better generalization to rare or unseen vocabulary. While statistical models remain efficient and strong baselines, **embedding-based sequence models offer superior performance** for nuanced, real-world emotion recognition in noisy social media domains.

```
Statistical Model Accuracy:   84.87%
Deep Learning Model Accuracy: 90.02%
```







3.2 Project summary and Reflections

This coursework designed, implemented, and evaluated a complete **end-to-end Natural Language Processing pipeline**for **multi-class emotion recognition** from social media text. Using a large-scale dataset of over **416,000 English-language tweets**, the study moved beyond binary sentiment analysis to identify six distinct emotions: *Joy, Sadness, Anger, Fear, Love,* and *Surprise*. The project demonstrates how modern NLP techniques can be applied to noisy, real-world data to extract psychologically meaningful signals.

A key contribution is the implementation of a **rigorous preprocessing strategy** tailored to the linguistic characteristics of social media. Challenges such as contractions, elongated words, HTML artifacts, and informal grammar were systematically addressed to ensure that models focused on emotionally relevant content rather than surface-level noise. This preprocessing pipeline played a critical role in stabilizing training and improving generalization across all models.

Four classification architectures were implemented and compared: **Multinomial Naive Bayes** as a statistical baseline, **Linear Support Vector Machine** as an enhanced classical model, a **Feed-Forward Neural Network** as a deep learning baseline, and a **Bi-Directional LSTM with pre-trained GloVe embeddings** as the most advanced architecture. The comparative evaluation revealed a clear progression in performance as models gained greater capacity to capture semantic meaning and contextual dependencies.

The findings confirm that **deep learning approaches outperform traditional bag-of-words-based statistical models** for complex emotion recognition tasks. In particular, the Bi-Directional LSTM achieved the highest overall performance, demonstrating the importance of sequential modeling and contextual awareness. The use of **transfer learning via GloVe embeddings** proved especially impactful, enabling better generalization to unseen and rare vocabulary. Overall, the results highlight that **data quality is as important as model complexity**, and that combining careful preprocessing with embedding-based deep learning yields robust performance in noisy social media environments.

- Real-World Applications

The emotion recognition system developed in this coursework demonstrates strong potential for real-world application across multiple domains by enabling scalable analysis of affective patterns in large volumes of informal text.

In **mental health monitoring**, the system could serve as a preliminary screening or triage tool, identifying users who consistently express emotions such as *Sadness* or *Fear*. While not intended to replace clinical diagnosis, it could support professionals by highlighting patterns of emotional distress for further review.

In **brand and customer experience management**, emotion-based classification allows organizations to prioritize urgent complaints by filtering social media mentions associated with emotions such as *Anger*. This enables more efficient resource allocation than relying solely on keyword matching or sentiment polarity, demonstrating how emotion-aware NLP systems can transform unstructured social media data into actionable insights.

- Limitations and Future Work

Despite strong performance, several limitations offer opportunities for improvement. A key limitation is the use of **static word embeddings**, which assign a fixed representation to each word regardless of context, leading to ambiguity for polysemous terms (e.g. *"bank"*). Future work could address this by adopting **Transformer-based models** such as *BERT*or *RoBERTa*, which generate contextualized embeddings and represent the current state of the art in NLP.

Another challenge is the prevalence of **irony and sarcasm** in social media, which can invert emotional meaning, particularly for *Surprise* and *Anger*. Incorporating a dedicated sarcasm detection component could improve robustness. Finally, more extensive **hyperparameter tuning**, using methods such as Grid Search or Bayesian Optimization, could further enhance performance and training stability.

References

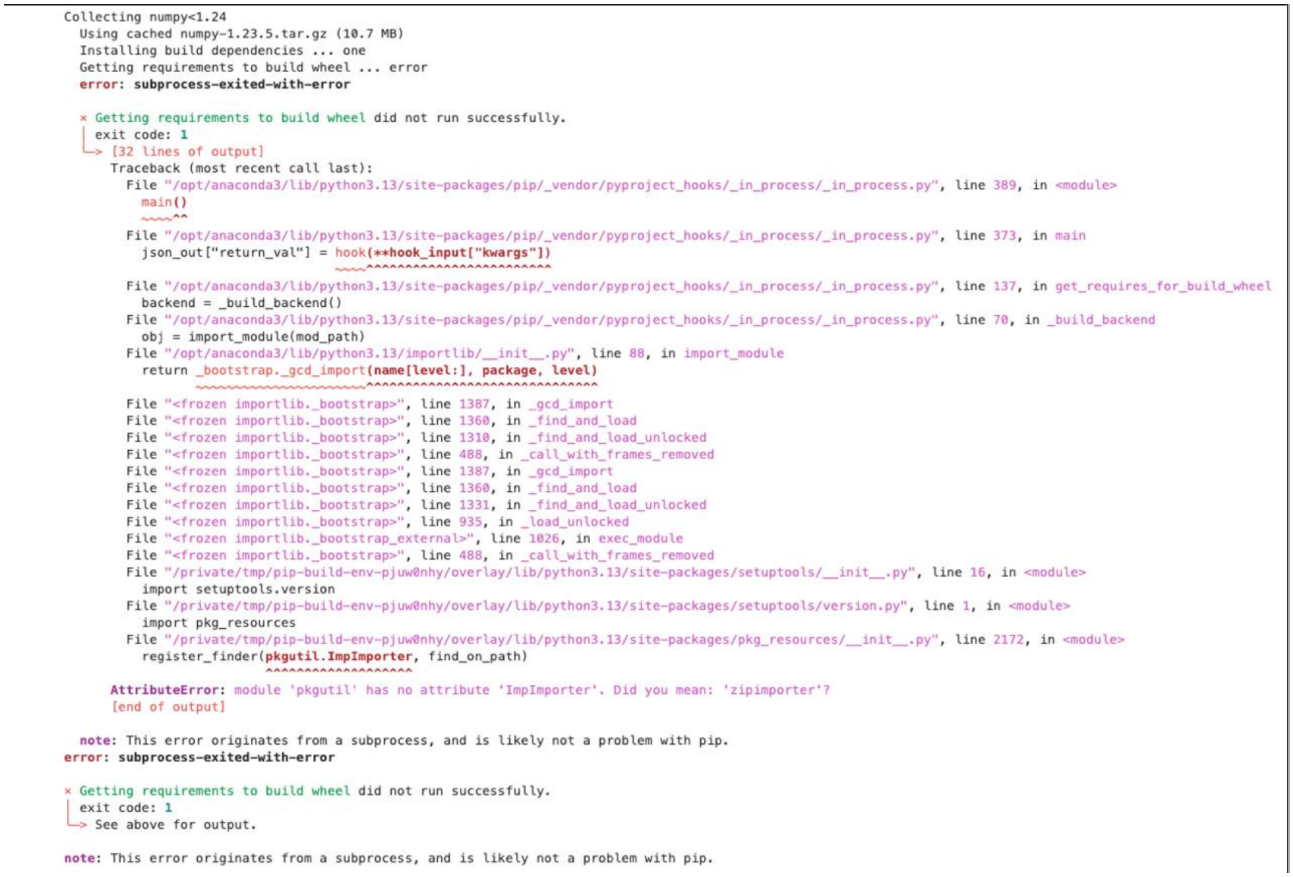
Elkomy, A. (2024) *Twitter Emotion Dataset: Unveiling the Emotional Tapestry of Social Media* [Data set]. Kaggle. Available at: https://www.kaggle.com/datasets/adhamelkomy/twitter-emotion-dataset/data

Code Appendix

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /opt/anaconda3/lib/python3.13/site-packages (2.20.0)
Requirement already satisfied: absl-py>=1.0.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (2.3.1)
Requirement already satisfied: astunparse>=1.6.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (24.2)
Requirement already satisfied: protobuf>=5.28.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (5.29.3)
Requirement already satisfied: requests<3,>=2.21.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (2.32.3)
Requirement already satisfied: setuptools in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (72.1.0)
Requirement already satisfied: six>=1.12.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (4.12.2)
Requirement already satisfied: wrapt>=1.11.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (1.17.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (1.76.0)
Requirement already satisfied: tensorboard~=2.20.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (3.13.0)
Requirement already satisfied: numpy>=1.26.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (2.1.3)
Requirement already satisfied: h5py>=3.11.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (3.12.1)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /opt/anaconda3/lib/python3.13/site-packages (from tensorflow) (0.5.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /opt/anaconda3/lib/python3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/anaconda3/lib/python3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/anaconda3/lib/python3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /opt/anaconda3/lib/python3.13/site-packages (from requests<3,>=2.21.0->tensorflow) (2025.4.26)
Requirement already satisfied: markdown>=2.6.8 in /opt/anaconda3/lib/python3.13/site-packages (from tensorboard~=2.20.0->tensorflow) (3.8)
Requirement already satisfied: pillow in /opt/anaconda3/lib/python3.13/site-packages (from tensorboard~=2.20.0->tensorflow) (11.1.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /opt/anaconda3/lib/python3.13/site-packages (from tensorboard~=2.20.0->tensorflow) (0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in /opt/anaconda3/lib/python3.13/site-packages (from tensorboard~=2.20.0->tensorflow) (3.1.3)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /opt/anaconda3/lib/python3.13/site-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)
Requirement already satisfied: rich in /opt/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /opt/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /opt/anaconda3/lib/python3.13/site-packages (from keras>=3.10.0->tensorflow) (0.18.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in /opt/anaconda3/lib/python3.13/site-packages (from werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /opt/anaconda3/lib/python3.13/site-packages (from rich->keras>=3.10.0->tensorflow) (2.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /opt/anaconda3/lib/python3.13/site-packages (from rich->keras>=3.10.0->tensorflow) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /opt/anaconda3/lib/python3.13/site-packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.0)
```

```
!pip install "numpy<1.24"
```



```
Collecting numpy<1.24
  Using cached numpy-1.23.5.tar.gz (10.7 MB)
  Installing build dependencies ... one
  Getting requirements to build wheel ... error
  error: subprocess-exited-with-error

  × Getting requirements to build wheel did not run successfully.
  │ exit code: 1
  └─> [32 lines of output]
      Traceback (most recent call last):
        File "/opt/anaconda3/lib/python3.13/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 389, in <module>
          main()
          ~~~~^^
        File "/opt/anaconda3/lib/python3.13/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 373, in main
          json_out["return_val"] = hook(**hook_input["kwargs"])
                                   ~~~~^^^^^^^^^^^^^^^^^^^^^^^^
        File "/opt/anaconda3/lib/python3.13/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 137, in get_requires_for_build_wheel
          backend = _build_backend()
        File "/opt/anaconda3/lib/python3.13/site-packages/pip/_vendor/pyproject_hooks/_in_process/_in_process.py", line 70, in _build_backend
          obj = import_module(mod_path)
        File "/opt/anaconda3/lib/python3.13/importlib/__init__.py", line 88, in import_module
          return _bootstrap._gcd_import(name[level:], package, level)
                 ~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
        File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
        File "<frozen importlib._bootstrap>", line 1310, in _find_and_load_unlocked
        File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
        File "<frozen importlib._bootstrap>", line 1387, in _gcd_import
        File "<frozen importlib._bootstrap>", line 1360, in _find_and_load
        File "<frozen importlib._bootstrap>", line 1331, in _find_and_load_unlocked
        File "<frozen importlib._bootstrap>", line 935, in _load_unlocked
        File "<frozen importlib._bootstrap_external>", line 1026, in exec_module
        File "<frozen importlib._bootstrap>", line 488, in _call_with_frames_removed
        File "/private/tmp/pip-build-env-pjuw0nhy/overlay/lib/python3.13/site-packages/setuptools/__init__.py", line 16, in <module>
          import setuptools.version
        File "/private/tmp/pip-build-env-pjuw0nhy/overlay/lib/python3.13/site-packages/setuptools/version.py", line 1, in <module>
          import pkg_resources
        File "/private/tmp/pip-build-env-pjuw0nhy/overlay/lib/python3.13/site-packages/pkg_resources/__init__.py", line 2172, in <module>
          register_finder(pkgutil.ImpImporter, find_on_path)
                          ^^^^^^^^^^^^^^^^^^^
      AttributeError: module 'pkgutil' has no attribute 'ImpImporter'. Did you mean: 'zipimporter'?
      [end of output]

    note: This error originates from a subprocess, and is likely not a problem with pip.
  error: subprocess-exited-with-error

  × Getting requirements to build wheel did not run successfully.
  │ exit code: 1
  └─> See above for output.

  note: This error originates from a subprocess, and is likely not a problem with pip.
```

```
!pip install --upgrade ml_dtypes
```

```
Requirement already satisfied: ml_dtypes in /opt/anaconda3/lib/python3.12/site-packages (0.3.2)
Collecting ml_dtypes
  Using cached ml_dtypes-0.5.4-cp312-cp312-macosx_10_13_universal2.whl.metadata (8.9 kB)
Requirement already satisfied: numpy>=1.21 in /opt/anaconda3/lib/python3.12/site-packages (from ml_dtypes) (1.2
6.4)
Using cached ml_dtypes-0.5.4-cp312-cp312-macosx_10_13_universal2.whl (676 kB)
Installing collected packages: ml_dtypes
  Attempting uninstall: ml_dtypes
    Found existing installation: ml-dtypes 0.3.2
    Uninstalling ml-dtypes-0.3.2:
      Successfully uninstalled ml-dtypes-0.3.2
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This
behaviour is the source of the following dependency conflicts.
tensorflow 2.16.1 requires ml-dtypes~=0.3.1, but you have ml-dtypes 0.5.4 which is incompatible.
Successfully installed ml_dtypes-0.5.4
```

```
!pip install numpy==1.26.4 tensorflow==2.16.1 ml_dtypes==0.3.2
```

```
Requirement already satisfied: numpy==1.26.4 in /opt/anaconda3/lib/python3.12/site-packages (1.26.4)
Requirement already satisfied: tensorflow==2.16.1 in /opt/anaconda3/lib/python3.12/site-packages (2.16.1)
Collecting ml_dtypes==0.3.2
  Using cached ml_dtypes-0.3.2-cp312-cp312-macosx_10_9_universal2.whl.metadata (20 kB)
Requirement already satisfied: absl-py>=1.0.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==
2.16.1) (2.3.1)
Requirement already satisfied: astunparse>=1.6.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflo
w==2.16.1) (1.6.3)
Requirement already satisfied: flatbuffers>=23.5.26 in /opt/anaconda3/lib/python3.12/site-packages (from tensor
flow==2.16.1) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /opt/anaconda3/lib/python3.12/site-packag
es (from tensorflow==2.16.1) (0.7.0)
Requirement already satisfied: google-pasta>=0.1.1 in /opt/anaconda3/lib/python3.12/site-packages (from tensorf
low==2.16.1) (0.2.0)
Requirement already satisfied: h5py>=3.10.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==2.
16.1) (3.11.0)
Requirement already satisfied: libclang>=13.0.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow
==2.16.1) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflo
w==2.16.1) (3.4.0)
Requirement already satisfied: packaging in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==2.16.
1) (24.1)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<5.0.0dev,>=3.20.3
in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==2.16.1) (4.25.8)
Requirement already satisfied: requests<3,>=2.21.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorf
low==2.16.1) (2.32.3)
Requirement already satisfied: setuptools in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==2.1
6.1) (75.1.0)
Requirement already satisfied: six>=1.12.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==2.1
6.1) (1.16.0)
Requirement already satisfied: termcolor>=1.1.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow
==2.16.1) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /opt/anaconda3/lib/python3.12/site-packages (from te
nsorflow==2.16.1) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /opt/anaconda3/lib/python3.12/site-packages (from tensorflow==
2.16.1) (1.14.1)
```

```
## importing the libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
from sklearn.model_selection import train_test_split
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
import html
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, confusion_matrix
import tensorflow as tf
from tensorflow.keras.layers import TextVectorization, Embedding,
GlobalAveragePooling1D, Dense, Dropout
from tensorflow.keras.models import Sequential
from tensorflow.keras.utils import to_categorical
from sklearn.preprocessing import LabelEncoder
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline
import os
import requests
import zipfile
```

```
import io
from tensorflow.keras.layers import Bidirectional, LSTM, Input




# 1. Load the dataset
df = pd.read_csv('emotions.csv')

# 2. Map the numerical labels to actual emotion names
emotion_map = {
    0: 'Sadness',
    1: 'Joy',
    2: 'Love',
    3: 'Anger',
    4: 'Fear',
    5: 'Surprise'
}

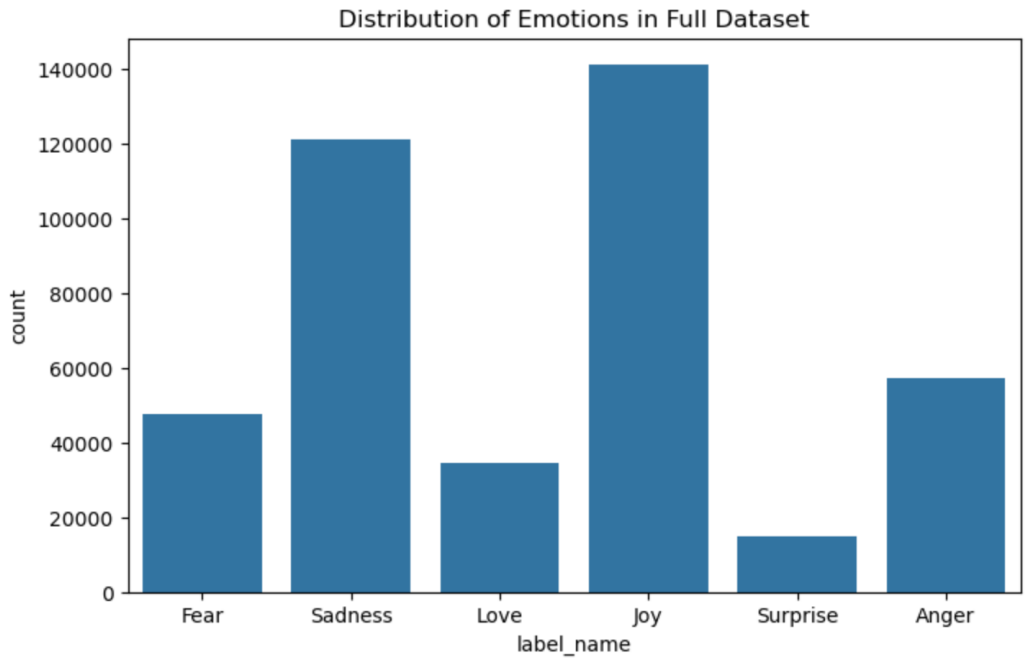
df['label_name'] = df['label'].map(emotion_map)

# 3. Use the FULL dataset


# 4. Show the data
print("Total Dataset Size:", len(df))
display(df.head())

# 5. Visualize the data
plt.figure(figsize=(8,5))
# using 'data=df' here to ensure the graph represents ALL 416,809 rows
sns.countplot(x='label_name', data=df)
plt.title('Distribution of Emotions in Full Dataset')
plt.show()
```

	text	label	label_name
0	i just feel really helpless and heavy hearted	4	Fear
1	ive enjoyed being able to slouch about relax a...	0	Sadness
2	i gave up my internship with the dmrg and am f...	4	Fear
3	i dont know i feel so lost	0	Sadness
4	i am a kindergarten teacher and i am thoroughl...	4	Fear



```
# Define a function that normalizes and sanitizes raw text for modeling
def clean_text(text):
    # Convert all characters to lowercase to reduce vocabulary size and
case-based duplicates
    text = text.lower()
    # Remove URL patterns (http/https/www) to avoid treating links as
meaningful tokens
    text = re.sub(r'http\S+|www\S+|https\S+', '', text,
flags=re.MULTILINE)
    # Remove @mentions and strip the '#' symbol from hashtags to keep
only the word content
    text = re.sub(r'\@\w+|\#', '', text)
    # Remove punctuation characters to simplify tokenization and prevent
punctuation-only tokens
    text = text.translate(str.maketrans('', '', string.punctuation))
    # Trim leading/trailing whitespace created by removals and
replacements
    text = text.strip()
    return text

# Apply the cleaning function to the dataset and store results in a new
column
print("Cleaning text... this might take a moment.")
df['clean_text'] = df['text'].apply(clean_text)

# Split features and labels into training and testing partitions
# Training receives 80% of samples, testing receives 20% for evaluation
# Stratification preserves label distribution across both splits
X_train, X_test, y_train, y_test = train_test_split(
    df['clean_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']  # Keeps class proportions consistent in train
and test sets
)

# Print basic diagnostics to show split sizes and confirm pipeline
completion
print("Data successfully cleaned and split!")
print(f"Training Data: {len(X_train)} rows")
print(f"Testing Data: {len(X_test)} rows")

# Display a before/after example to verify cleaning behavior on a real
sample
print("\nExample - Original:", df['text'].iloc[5])
print("Example - Cleaned:", df['clean_text'].iloc[5])
```

```
Cleaning text... this might take a moment.
Data successfully cleaned and split!
Training Data: 333447 rows
Testing Data: 83362 rows

Example - Original: i was beginning to feel quite disheartened
Example - Cleaned: i was beginning to feel quite disheartened
```

```
# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt_tab')

#  Define the heavy-duty cleaning function
def advanced_clean_text(text):
    #  Basic cleaning (Lowercase, remove URLs/Punctuation)
    text = text.lower()
    text = re.sub(r'http\S+|www\S+|https\S+', '', text,
flags=re.MULTILINE)
    text = re.sub(r'\@\w+|\#', '', text)
    text = text.translate(str.maketrans('', '', string.punctuation))

    #  Tokenization (Split into words)
    tokens = word_tokenize(text)

    #  Remove Stopwords (e.g., "is", "the", "and")
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    #  Lemmatization (Convert to root form: "running" -> "run")
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]

    #  Join back into a single string
    return " ".join(tokens)

# Apply the function
print("Starting advanced preprocessing (Tokenization &
Lemmatization)...")
df['processed_text'] = df['text'].apply(advanced_clean_text)

#  Split the data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    df['processed_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']
)

print("\nSuccess! Data is cleaned, lemmatized, and split.")
print("-" * 50)
print("Original Example: ", df['text'].iloc[0])
print("Processed Example:", df['processed_text'].iloc[0])
```

```
Starting advanced preprocessing (Tokenization & Lemmatization)...

Success! Data is cleaned, lemmatized, and split.
--------------------------------------------------
Original Example:  i just feel really helpless and heavy hearted
Processed Example: feel really helpless heavy hearted
```

```
# Download necessary NLTK data
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt_tab')

# Define helper tools
# A dictionary to fix contractions (e.g., "don't" -> "do not")
contractions_dict = {
    "im": "i am", "i'm": "i am", "dont": "do not", "don't": "do not",
    "cant": "cannot", "can't": "cannot", "wont": "will not", "won't":
"will not",
    "ive": "i have", "i've": "i have", "id": "i would", "i'd": "i would",
    "youre": "you are", "you're": "you are", "thats": "that is",
"that's": "that is",
    "didnt": "did not", "didn't": "did not", "wouldnt": "would not",
"wouldn't": "would not"
}

def expand_contractions(text):
    for word, replacement in contractions_dict.items():
        # Replace the contraction with the full form
        text = text.replace(word, replacement)
    return text

def remove_elongated(text):
    # Replaces characters repeating more than 2 times with 2 characters
    # Example: "haaaappy" -> "haappy" (The lemmatizer can often fix
"haappy" -> "happy")
    return re.sub(r'(.)\1{2,}', r'\1\1', text)

#  Define the Master Cleaning Function
def super_clean_text(text):
    #  Decode HTML entities (e.g., "&amp;" -> "&")
    text = html.unescape(text)

    #  Lowercase
    text = text.lower()

    #  Expand Contractions (BEFORE removing punctuation)
    text = expand_contractions(text)

    # Remove URLs, Handles (@user), and Hashtags (#)
    text = re.sub(r'http\S+|www\S+|https\S+', '', text,
flags=re.MULTILINE)
    text = re.sub(r'\@\w+|\#', '', text)

    #  Remove Numbers (Digits)
    text = re.sub(r'\d+', '', text)

    #  Remove Elongated Words (soooo -> soo)
    text = remove_elongated(text)

    #  Remove Punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    #  Tokenization
    tokens = word_tokenize(text)
```

```
    #  Remove Stopwords & J. Lemmatization
    stop_words = set(stopwords.words('english'))
    lemmatizer = WordNetLemmatizer()

    clean_tokens = []
    for word in tokens:
        if word not in stop_words:
            clean_word = lemmatizer.lemmatize(word)
            clean_tokens.append(clean_word)

    return " ".join(clean_tokens)

#  Apply the function
print("Starting SUPER advanced preprocessing...")
df['processed_text'] = df['text'].apply(super_clean_text)

#  Split the data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    df['processed_text'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']
)

print("\nSuccess! Data is cleaned, contracted, shortened, and split.")
print("-" * 50)
print("Original Example: ", df['text'].iloc[0])
print("Processed Example:", df['processed_text'].iloc[0])
```

```
[nltk_data]    Package punkt_tab is already up-to-date!

Success! Data is cleaned, contracted, shortened, and split.
--------------------------------------------------
Original Example:  i just feel really helpless and heavy hearted
Processed Example: feel really helpless heavy hearted
```

```
print("Step 1: Training Statistical Model (Naive Bayes)...")

# Define the Pipeline
model_stat = Pipeline([
    ('tfidf', TfidfVectorizer(max_features=5000)),
    ('clf', MultinomialNB())
])

# Train using the raw text (X_train)
model_stat.fit(X_train, y_train)
print("✓ Statistical Model trained successfully!")
```

```
Step 1: Training Statistical Model (Naive Bayes)...
✓ Statistical Model trained successfully!
```

```
print("--- EVALUATING STATISTICAL MODEL (Naive Bayes) ---")

#  Get Predictions
# We use X_test (raw text) for the statistical model
y_pred_stat = model_stat.predict(X_test)

#  Print Detailed Report (Accuracy, Precision, Recall, F1)
print("\nClassification Report:")
print(classification_report(y_test, y_pred_stat,
                            target_names=['Sadness', 'Joy', 'Love',
'Anger', 'Fear', 'Surprise']))

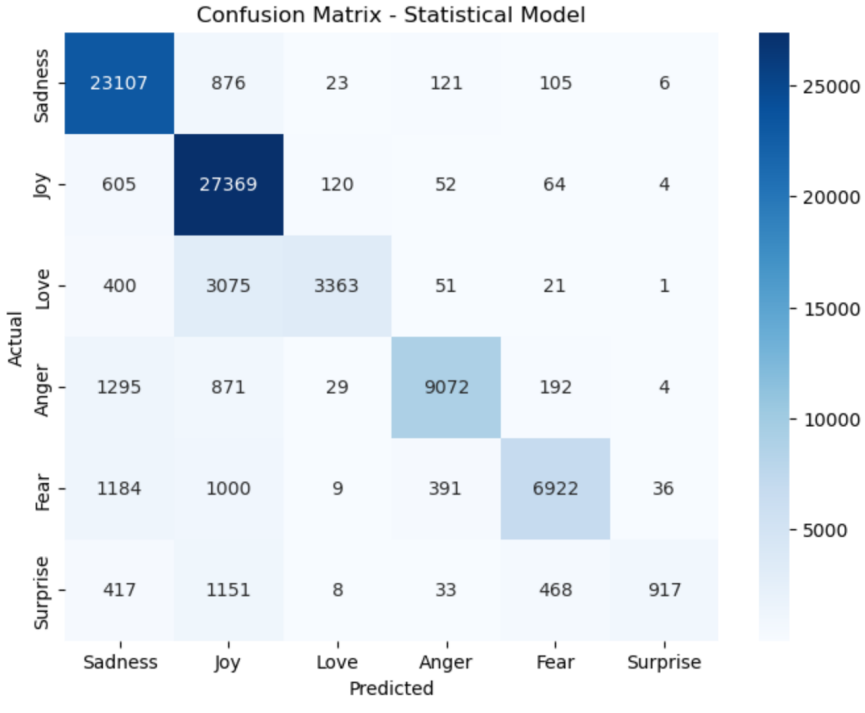
#  Plot Confusion Matrix
plt.figure(figsize=(8,6))
sns.heatmap(confusion_matrix(y_test, y_pred_stat), annot=True, fmt='d',
cmap='Blues',
            xticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear',
'Surprise'],
            yticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear',
'Surprise'])
plt.title('Confusion Matrix - Statistical Model')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
--- EVALUATING STATISTICAL MODEL (Naive Bayes) ---

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.86      0.95      0.90     24238
         Joy       0.80      0.97      0.88     28214
        Love       0.95      0.49      0.64      6911
       Anger       0.93      0.79      0.86     11463
        Fear       0.89      0.73      0.80      9542
    Surprise       0.95      0.31      0.46      2994

    accuracy                           0.85     83362
   macro avg       0.90      0.71      0.76     83362
weighted avg       0.86      0.85      0.84     83362
```



```
print("Step 2: Training Deep Learning Model...")

# Encode string/categorical labels into integer class IDs for neural
network training
le = LabelEncoder()
y_train_enc = le.fit_transform(y_train)
y_test_enc = le.transform(y_test)

# Convert integer class IDs into one-hot vectors for categorical cross-
entropy loss
y_train_cat = to_categorical(y_train_enc)
y_test_cat = to_categorical(y_test_enc)

# Convert text collections into TensorFlow string tensors to ensure
consistent input types
X_train_safe = tf.convert_to_tensor(list(X_train))
X_test_safe = tf.convert_to_tensor(list(X_test))

# Configure a text vectorization layer that maps raw strings to fixed-
length integer token sequences
vocab_size = 10000
vectorizer = TextVectorization(max_tokens=vocab_size,
output_sequence_length=50)

# Build the vocabulary and tokenization rules from training text only to
avoid test leakage
vectorizer.adapt(X_train_safe)

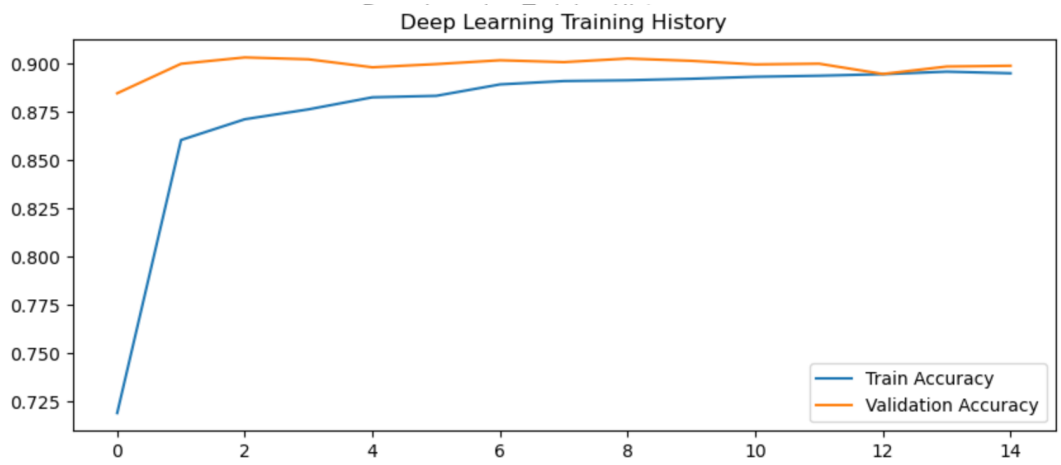
# Create a lightweight text classification model:
# - Vectorization converts strings to token IDs
# - Embedding learns dense token representations
# - Global pooling aggregates token embeddings into a single sentence
vector
# - Dense + dropout add nonlinear capacity and regularization
# - Final softmax outputs class probabilities
model_dl = Sequential([
    vectorizer,
    Embedding(input_dim=vocab_size, output_dim=64),
    GlobalAveragePooling1D(),
    Dense(24, activation='relu'),
    Dropout(0.5),
    Dense(6, activation='softmax')
])

# Compile the model with Adam optimizer, categorical cross-entropy for
one-hot labels, and accuracy tracking
model_dl.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Train the network on the training set and evaluate each epoch against
the test split as validation data
history = model_dl.fit(
    X_train_safe, y_train_cat,
    epochs=15,
    batch_size=32,
    validation_data=(X_test_safe, y_test_cat),
    verbose=1
)
```

```
# Visualize training and validation accuracy across epochs to inspect
learning progress and potential overfitting
plt.figure(figsize=(10, 4))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Deep Learning Training History')
plt.legend()
plt.show()
```

```
Step 2: Training Deep Learning Model...
Epoch 1/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 28s 3ms/step - accuracy: 0.7189 - loss: 0.7733 - val_accuracy: 0.8845 - val_loss: 0.3116
Epoch 2/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8603 - loss: 0.3960 - val_accuracy: 0.8998 - val_loss: 0.2390
Epoch 3/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 30s 3ms/step - accuracy: 0.8711 - loss: 0.3415 - val_accuracy: 0.9031 - val_loss: 0.2301
Epoch 4/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8762 - loss: 0.3127 - val_accuracy: 0.9021 - val_loss: 0.2238
Epoch 5/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8825 - loss: 0.2863 - val_accuracy: 0.8980 - val_loss: 0.2478
Epoch 6/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8832 - loss: 0.2767 - val_accuracy: 0.8997 - val_loss: 0.2136
Epoch 7/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 28s 3ms/step - accuracy: 0.8891 - loss: 0.2596 - val_accuracy: 0.9016 - val_loss: 0.2210
Epoch 8/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8909 - loss: 0.2545 - val_accuracy: 0.9007 - val_loss: 0.2123
Epoch 9/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8913 - loss: 0.2492 - val_accuracy: 0.9025 - val_loss: 0.2113
Epoch 10/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8920 - loss: 0.2455 - val_accuracy: 0.9013 - val_loss: 0.2110
Epoch 11/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8931 - loss: 0.2405 - val_accuracy: 0.8995 - val_loss: 0.2278
Epoch 12/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 30s 3ms/step - accuracy: 0.8936 - loss: 0.2387 - val_accuracy: 0.8999 - val_loss: 0.2123
Epoch 13/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8944 - loss: 0.2365 - val_accuracy: 0.8945 - val_loss: 0.2199
Epoch 14/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8957 - loss: 0.2335 - val_accuracy: 0.8984 - val_loss: 0.2338
Epoch 15/15
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 29s 3ms/step - accuracy: 0.8949 - loss: 0.2325 - val_accuracy: 0.8988 - val_loss: 0.2457
```



```
print("--- EVALUATING DEEP LEARNING MODEL ---")

# Generate class probability predictions for the test set using the
trained neural network
y_pred_probs = model_dl.predict(X_test_safe)

# Convert probability vectors into discrete class indices by selecting
the highest-probability class
y_pred_dl = np.argmax(y_pred_probs, axis=1)

# Produce precision/recall/F1 metrics by comparing encoded ground-truth
labels against predicted class indices
print("\nClassification Report:")
print(classification_report(
    y_test_enc,
    y_pred_dl,
    target_names=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
))

# Build a confusion matrix to summarize how often each true class is
predicted as each class
cm = confusion_matrix(y_test_enc, y_pred_dl)

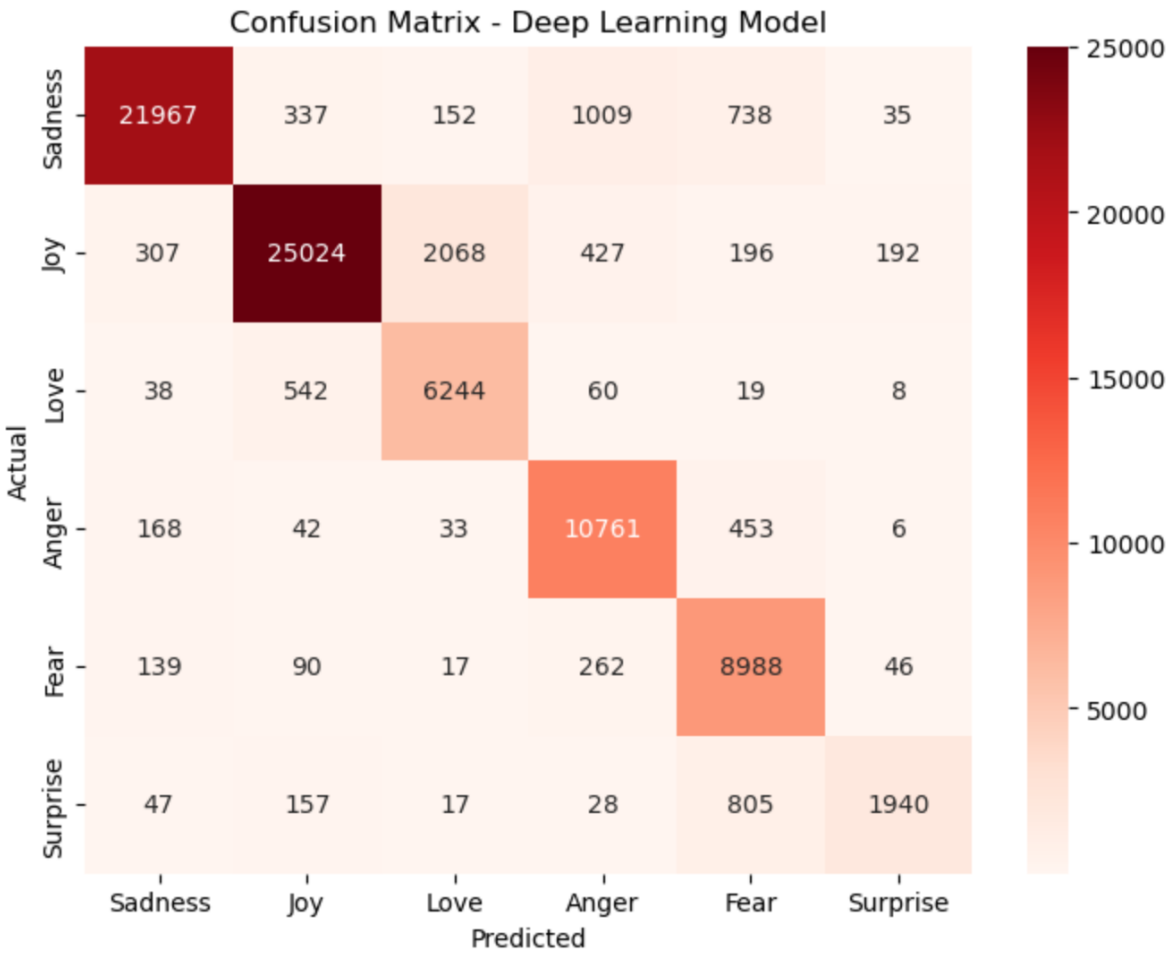
# Visualize the confusion matrix as a heatmap for quick inspection of
common confusions
plt.figure(figsize=(8, 6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Reds',
    xticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'],
    yticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
)
plt.title('Confusion Matrix - Deep Learning Model')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
--- EVALUATING DEEP LEARNING MODEL ---
2606/2606 ━━━━━━━━━━━━━━━━━━━━ 1s 414us/step

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.97      0.91      0.94     24238
         Joy       0.96      0.89      0.92     28214
        Love       0.73      0.90      0.81      6911
       Anger       0.86      0.94      0.90     11463
        Fear       0.80      0.94      0.87      9542
    Surprise       0.87      0.65      0.74      2994

    accuracy                           0.90     83362
   macro avg       0.86      0.87      0.86     83362
weighted avg       0.91      0.90      0.90     83362
```




```
print("Step 3: Generating Final Evaluation...")

# Generate probability outputs for each class on the test set using the
trained neural network
y_pred_probs = model_dl.predict(X_test_safe)

# Convert probability vectors into predicted class indices by selecting
the maximum probability per sample
y_pred_dl = np.argmax(y_pred_probs, axis=1)

# Display per-class precision, recall, and F1-score by comparing encoded
true labels against predicted indices
print("\n--- Deep Learning Classification Report ---")
print(classification_report(
    y_test_enc,
    y_pred_dl,
    target_names=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
))

# Compute the confusion matrix to summarize correct predictions and
class-to-class confusions
cm = confusion_matrix(y_test_enc, y_pred_dl)

# Render the confusion matrix as a heatmap for a visual view of
prediction distribution across classes
plt.figure(figsize=(8,6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
```

```
    cmap='Reds',
    xticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'],
    yticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
)
plt.title('Confusion Matrix - Deep Learning Model')
plt.show()

# Measure statistical model accuracy on the raw test split to provide a
baseline comparison
acc_stat = model_stat.score(X_test, y_test)

# Evaluate the deep learning model on the tensor-based test inputs and
one-hot labels
loss_dl, acc_dl = model_dl.evaluate(X_test_safe, y_test_cat, verbose=0)

# Print both accuracy values using percentage formatting for easy side-
by-side comparison
print(f"\nStatistical Model Accuracy:   {acc_stat:.2%}")
print(f"Deep Learning Model Accuracy: {acc_dl:.2%}")

# Assemble results into a small table to support plotting and optional
exporting
results = pd.DataFrame({
    'Model': ['Statistical (Naive Bayes)', 'Deep Learning (Neural Net)'],
    'Accuracy': [acc_stat, acc_dl]
})

# Plot a bar chart comparing accuracy values across the two model types
plt.figure(figsize=(8, 5))
sns.barplot(x='Model', y='Accuracy', data=results, palette='viridis')
plt.ylim(0, 1)
plt.title('Final Comparison: Statistical vs. Embedding Model')
plt.ylabel('Accuracy Score')

# Annotate each bar with its formatted accuracy value for readability
without hovering
for index, row in results.iterrows():
    plt.text(index, row.Accuracy + 0.02, f"{row.Accuracy:.2%}",
ha='center', fontweight='bold')

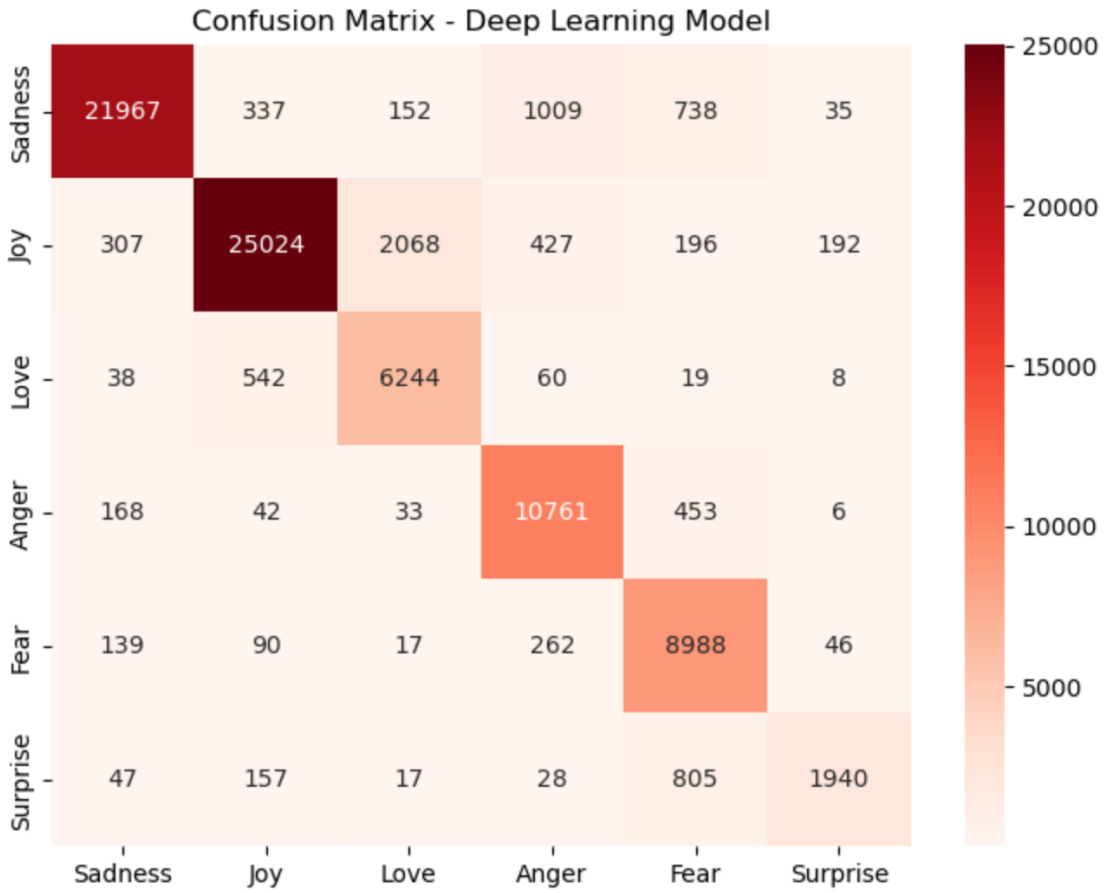
plt.show()
```

```
Step 3: Generating Final Evaluation...
2606/2606 ━━━━━━━━━━━━━━━━━━━━ 1s 406us/step

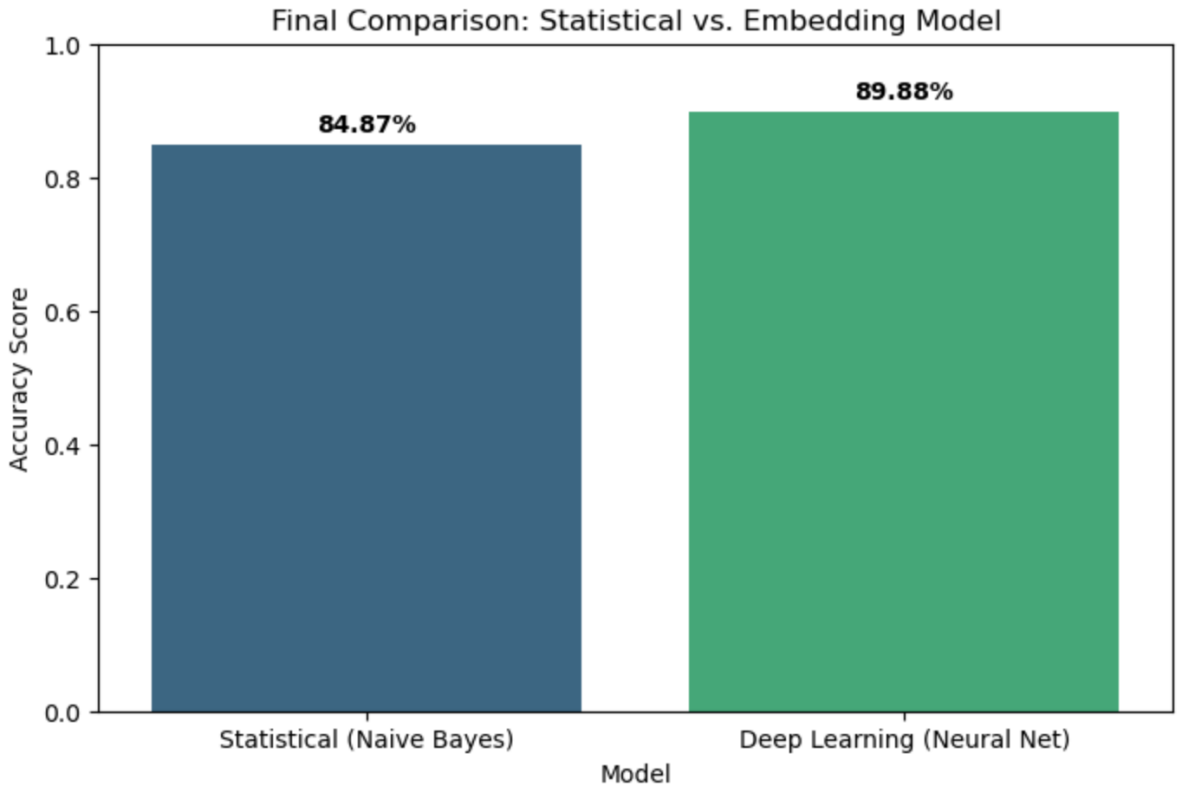
--- Deep Learning Classification Report ---
              precision    recall  f1-score   support

     Sadness       0.97      0.91      0.94     24238
         Joy       0.96      0.89      0.92     28214
        Love       0.73      0.90      0.81      6911
       Anger       0.86      0.94      0.90     11463
        Fear       0.80      0.94      0.87      9542
    Surprise       0.87      0.65      0.74      2994

    accuracy                           0.90     83362
   macro avg       0.86      0.87      0.86     83362
weighted avg       0.91      0.90      0.90     83362
```




```
Statistical Model Accuracy:   84.87%
Deep Learning Model Accuracy: 89.88%
```



```
print("--- Training New Statistical Model: Linear SVM ---")

# Create a text classification pipeline that transforms raw text into TF-IDF features,
# then applies a linear Support Vector Machine classifier on the resulting sparse vectors
model_svm = Pipeline([
    # Convert input text into a fixed-size TF-IDF feature space using the most frequent terms
    ('tfidf', TfidfVectorizer(max_features=5000)),
    # Train a linear SVM classifier; dual=False is suited for common sparse text settings and improves compatibility
    ('clf', LinearSVC(dual=False, random_state=42, max_iter=10000))
])

# Fit the pipeline end-to-end so TF-IDF statistics and the classifier are learned from the training split
model_svm.fit(X_train, y_train)

# Compute accuracy on the held-out test set to estimate generalization performance
acc_svm = model_svm.score(X_test, y_test)
print(f"✓ SVM Model trained! Accuracy: {acc_svm:.2%}")
```

```
--- Training New Statistical Model: Linear SVM ---
✓ SVM Model trained! Accuracy: 89.89%
```

```
print("--- DETAILED RESULTS: LINEAR SVM MODEL ---")

# Generate predicted class labels for the test split using the trained SVM pipeline
y_pred_svm = model_svm.predict(X_test)

# Display precision, recall, and F1-score per emotion by comparing true labels against model predictions
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred_svm,
    target_names=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
))

# Compute a confusion matrix to summarize correct predictions and common misclassifications
cm = confusion_matrix(y_test, y_pred_svm)

# Visualize the confusion matrix as a heatmap to make error patterns easy to spot
plt.figure(figsize=(8,6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Greens',
    xticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'],
    yticklabels=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
)
```

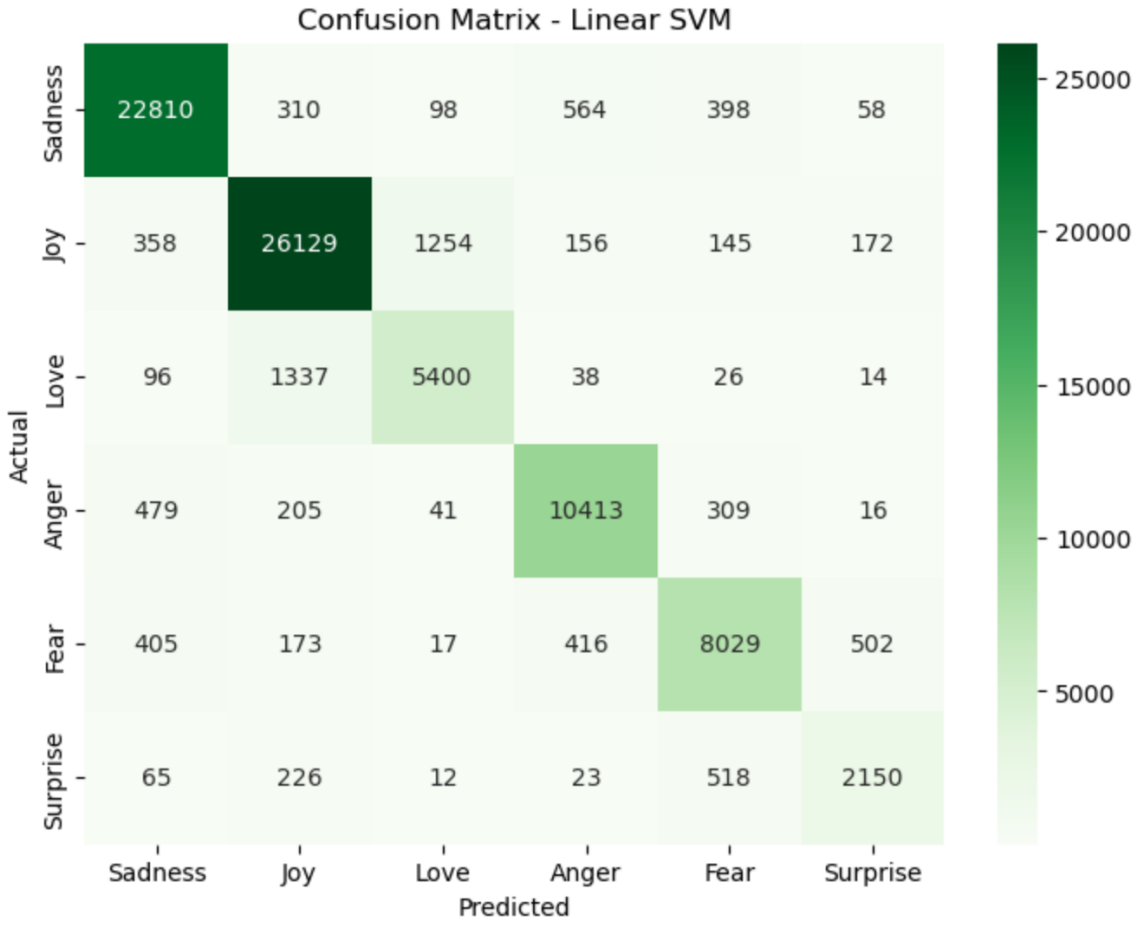
```
plt.title('Confusion Matrix - Linear SVM')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

```
--- DETAILED RESULTS: LINEAR SVM MODEL ---

Classification Report:
              precision    recall  f1-score   support

     Sadness       0.94      0.94      0.94     24238
         Joy       0.92      0.93      0.92     28214
        Love       0.79      0.78      0.79      6911
       Anger       0.90      0.91      0.90     11463
        Fear       0.85      0.84      0.85      9542
    Surprise       0.74      0.72      0.73      2994

    accuracy                           0.90     83362
   macro avg       0.86      0.85      0.85     83362
weighted avg       0.90      0.90      0.90     83362
```



```
print("--- Setting up GloVe Embeddings ---")

# Define filenames and the remote source for the pre-trained GloVe
embedding package
glove_file = 'glove.6B.100d.txt'
glove_zip = 'glove.6B.zip'
url = "http://nlp.stanford.edu/data/glove.6B.zip"

# Download and extract the required embedding text file only when it is
not already present locally
if not os.path.exists(glove_file):
    # Download the zip archive only when it is not already present
locally
    if not os.path.exists(glove_zip):
        print("Downloading GloVe embeddings (822MB)... This will take a
few minutes.")
        # Stream the download to avoid loading the full archive into
memory at once
        response = requests.get(url, stream=True)
        if response.status_code == 200:
            # Write the streamed bytes to disk in chunks for stability on
large files
            with open(glove_zip, 'wb') as f:
                for chunk in response.iter_content(chunk_size=1024):
                    if chunk:
                        f.write(chunk)
            print("Download complete.")
        else:
            # Report failure to download while keeping the script flow
intact
            print("Error downloading file.")

    # Extract only the needed embedding file from the zip archive
    print("Unzipping GloVe...")
    with zipfile.ZipFile(glove_zip, 'r') as zip_ref:
        zip_ref.extract(glove_file)
    print("Unzip complete!")
else:
    # Skip download/extraction when the target embedding file already
exists
    print("GloVe file already exists.")

# Load GloVe vectors into a dictionary mapping each word to its numeric
embedding vector
print("Loading word vectors...")
embeddings_index = {}
with open(glove_file, encoding="utf-8") as f:
    for line in f:
        # Split each line into a token followed by embedding values
        values = line.split()
        word = values[0]
        # Convert the numeric part into a float32 numpy array for
efficient storage and math
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs

print(f"Found {len(embeddings_index)} word vectors in GloVe.")
```

```
# Build an embedding matrix aligned to the current text vocabulary used
by the vectorizer
voc = vectorizer.get_vocabulary()
word_index = dict(zip(voc, range(len(voc))))

# Reserve extra rows to allow space for special tokens and index safety
num_tokens = len(voc) + 2
embedding_dim = 100

# Track vocabulary coverage by counting words that do or do not appear in
the pre-trained set
hits = 0
misses = 0

# Initialize the embedding matrix with zeros; missing tokens remain as
zero vectors
embedding_matrix = np.zeros((num_tokens, embedding_dim))

# Fill embedding rows for vocabulary tokens that exist in the GloVe
dictionary
for word, i in word_index.items():
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        embedding_matrix[i] = embedding_vector
        hits += 1
    else:
        misses += 1

print(f"Converted {hits} words. ({misses} misses)")
```

```
--- Setting up GloVe Embeddings ---
GloVe file already exists.
Loading word vectors...
Found 400000 word vectors in GloVe.
Converted 9553 words. (447 misses)
```

```
print("--- Training Advanced Deep Learning Model (LSTM + GloVe) ---")

# Build a text classification model that combines integer tokenization,
fixed GloVe embeddings,
# and a bidirectional recurrent layer to capture word order and context
from both directions
model_glove = Sequential([
    # Convert raw text strings into fixed-length integer token sequences
using the previously adapted vectorizer
    vectorizer,

    # Map token IDs to dense vectors using the pre-trained GloVe
embedding matrix
    # The embedding layer is frozen so pre-trained semantic structure is
preserved during training
    Embedding(
        input_dim=num_tokens,
        output_dim=embedding_dim,

embeddings_initializer=tf.keras.initializers.Constant(embedding_matrix),
        trainable=False
    ),
```

```
    # Process the embedding sequence with an LSTM that reads forward and
backward to capture context on both sides
    # return_sequences=False outputs a single vector summary for the
whole sequence
    Bidirectional(LSTM(64, return_sequences=False)),

    # Apply fully connected layers to transform the sequence
representation into class decision features
    Dense(64, activation='relu'),
    Dropout(0.5),

    # Output a probability distribution across the emotion classes using
softmax activation
    Dense(6, activation='softmax')
])

# Configure training with Adam optimization, categorical cross-entropy
for one-hot labels, and accuracy reporting
model_glove.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Train the model using tensor-safe text inputs and one-hot encoded
labels, validating on the held-out test split
history_glove = model_glove.fit(
    X_train_safe, y_train_cat,
    epochs=10,
    batch_size=32,
    validation_data=(X_test_safe, y_test_cat),
    verbose=1
)
```

```
--- Training Advanced Deep Learning Model (LSTM + GloVe) ---
Epoch 1/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 248s 24ms/step - accuracy: 0.8520 - loss: 0.3992 - val_accuracy: 0.9039 - val_loss: 0.2140
Epoch 2/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 250s 24ms/step - accuracy: 0.9063 - loss: 0.2119 - val_accuracy: 0.9096 - val_loss: 0.1891
Epoch 3/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 250s 24ms/step - accuracy: 0.9116 - loss: 0.1871 - val_accuracy: 0.9135 - val_loss: 0.1812
Epoch 4/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 251s 24ms/step - accuracy: 0.9141 - loss: 0.1763 - val_accuracy: 0.9124 - val_loss: 0.1786
Epoch 5/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 254s 24ms/step - accuracy: 0.9157 - loss: 0.1694 - val_accuracy: 0.9151 - val_loss: 0.1700
Epoch 6/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 299s 29ms/step - accuracy: 0.9168 - loss: 0.1639 - val_accuracy: 0.9139 - val_loss: 0.1790
Epoch 7/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 377s 36ms/step - accuracy: 0.9173 - loss: 0.1611 - val_accuracy: 0.9151 - val_loss: 0.1754
Epoch 8/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 388s 37ms/step - accuracy: 0.9183 - loss: 0.1574 - val_accuracy: 0.9159 - val_loss: 0.1713
Epoch 9/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 420s 40ms/step - accuracy: 0.9187 - loss: 0.1549 - val_accuracy: 0.9146 - val_loss: 0.1766
Epoch 10/10
10421/10421 ━━━━━━━━━━━━━━━━━━━━ 374s 36ms/step - accuracy: 0.9188 - loss: 0.1530 - val_accuracy: 0.9159 - val_loss: 0.1740
```

```
print("--- LSTM + GloVe Model: Confusion Matrix & Metrics ---")

# Generate class probability predictions for the test split using the
trained LSTM + GloVe network
y_pred_probs_lstm = model_glove.predict(X_test_safe)

# Convert probability vectors into predicted class indices by selecting
the maximum probability per sample
y_pred_lstm = np.argmax(y_pred_probs_lstm, axis=1)

# Compute the confusion matrix to capture correct predictions and class-
to-class misclassifications
cm_lstm = confusion_matrix(y_test_enc, y_pred_lstm)
```

```
# Wrap the confusion matrix in a labeled DataFrame to keep class names
attached to rows and columns
cm_df = pd.DataFrame(
    cm_lstm,
    index=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise'],
    columns=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
)

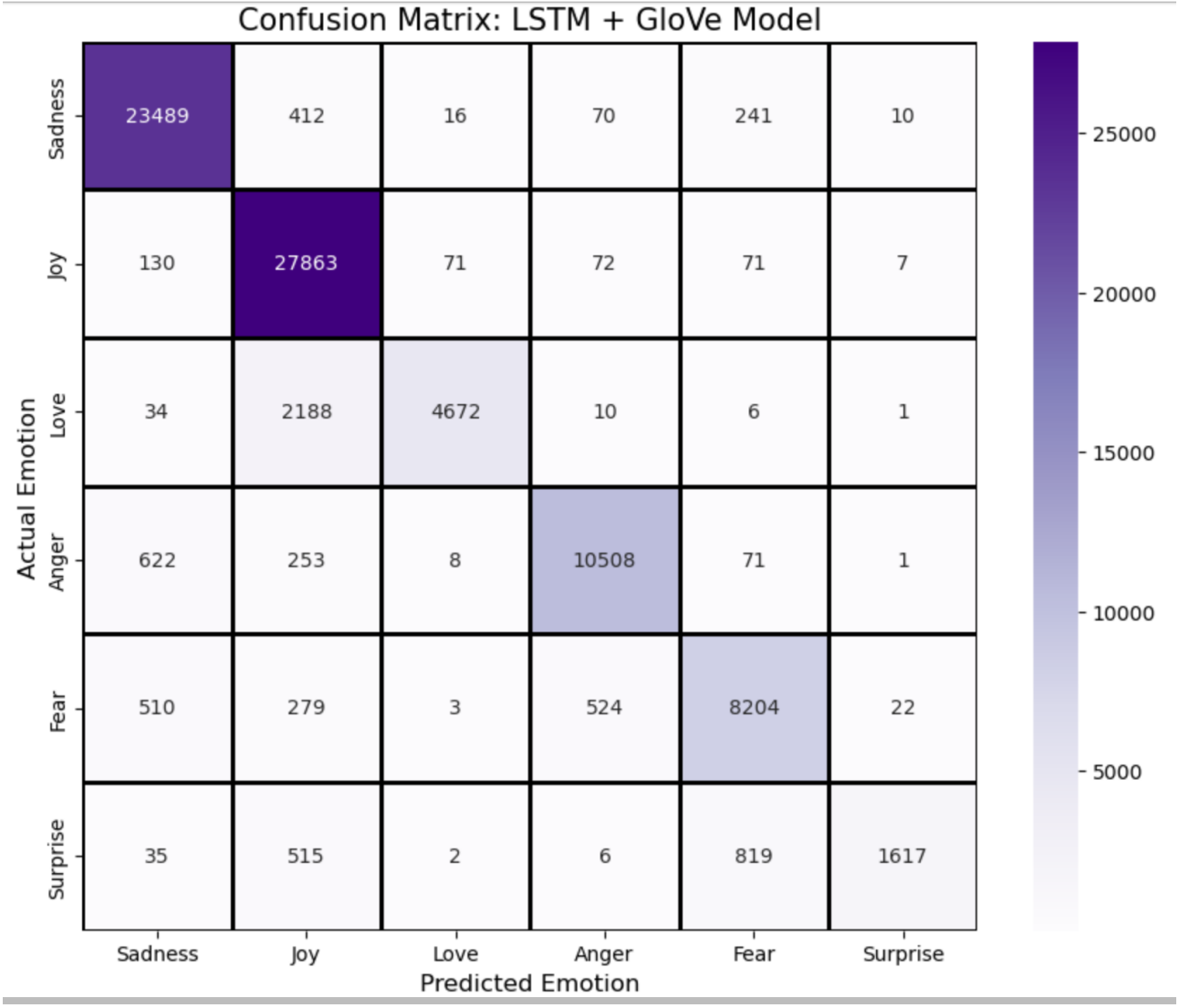
# Print the raw confusion matrix counts for direct reporting and manual
verification
print("\nFull Confusion Matrix Data (Numerical):")
print(cm_df)

# Render the confusion matrix as a heatmap to make dominant patterns and
frequent confusions visually obvious
plt.figure(figsize=(10, 8))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Purples', linewidths=1,
linecolor='black')
plt.title('Confusion Matrix: LSTM + GloVe Model', fontsize=15)
plt.ylabel('Actual Emotion', fontsize=12)
plt.xlabel('Predicted Emotion', fontsize=12)
plt.show()

# Print precision, recall, and F1-score per class by comparing encoded
ground truth against predicted class indices
print("\nDetailed Classification Report:")
print(classification_report(
    y_test_enc,
    y_pred_lstm,
    target_names=['Sadness', 'Joy', 'Love', 'Anger', 'Fear', 'Surprise']
))
```

```
--- LSTM + GloVe Model: Confusion Matrix & Metrics ---
2606/2606 ━━━━━━━━━━━━━━━━━━━━ 29s 11ms/step

Full Confusion Matrix Data (Numerical):
          Sadness    Joy  Love  Anger  Fear  Surprise
Sadness     23489    412    16     70   241        10
Joy           130  27863    71     72    71         7
Love           34   2188  4672     10     6         1
Anger         622    253     8  10508    71         1
Fear          510    279     3    524  8204        22
Surprise       35    515     2      6   819      1617
```



Detailed Classification Report:

	precision	recall	f1-score	support
Sadness	0.95	0.97	0.96	24238
Joy	0.88	0.99	0.93	28214
Love	0.98	0.68	0.80	6911
Anger	0.94	0.92	0.93	11463
Fear	0.87	0.86	0.87	9542
Surprise	0.98	0.54	0.70	2994
accuracy			0.92	83362
macro avg	0.93	0.82	0.86	83362
weighted avg	0.92	0.92	0.91	83362

```
print("--- FINAL PROJECT COMPARISON ---")

# Gather all accuracies
# Statistical Models
acc_nb = model_stat.score(X_test, y_test) # Your original NB model
acc_svm = model_svm.score(X_test, y_test) # The new SVM model

# Deep Learning Models
```

```
_, acc_simple_dl = model_dl.evaluate(X_test_safe, y_test_cat, verbose=0)
# Original DL
_, acc_glove_dl = model_glove.evaluate(X_test_safe, y_test_cat,
verbose=0) # New GloVe DL

print(f"Naive Bayes Accuracy:      {acc_nb:.2%}")
print(f"SVM Accuracy:              {acc_svm:.2%}")
print(f"Simple Neural Net Acc:     {acc_simple_dl:.2%}")
print(f"GloVe + LSTM Accuracy:     {acc_glove_dl:.2%}")

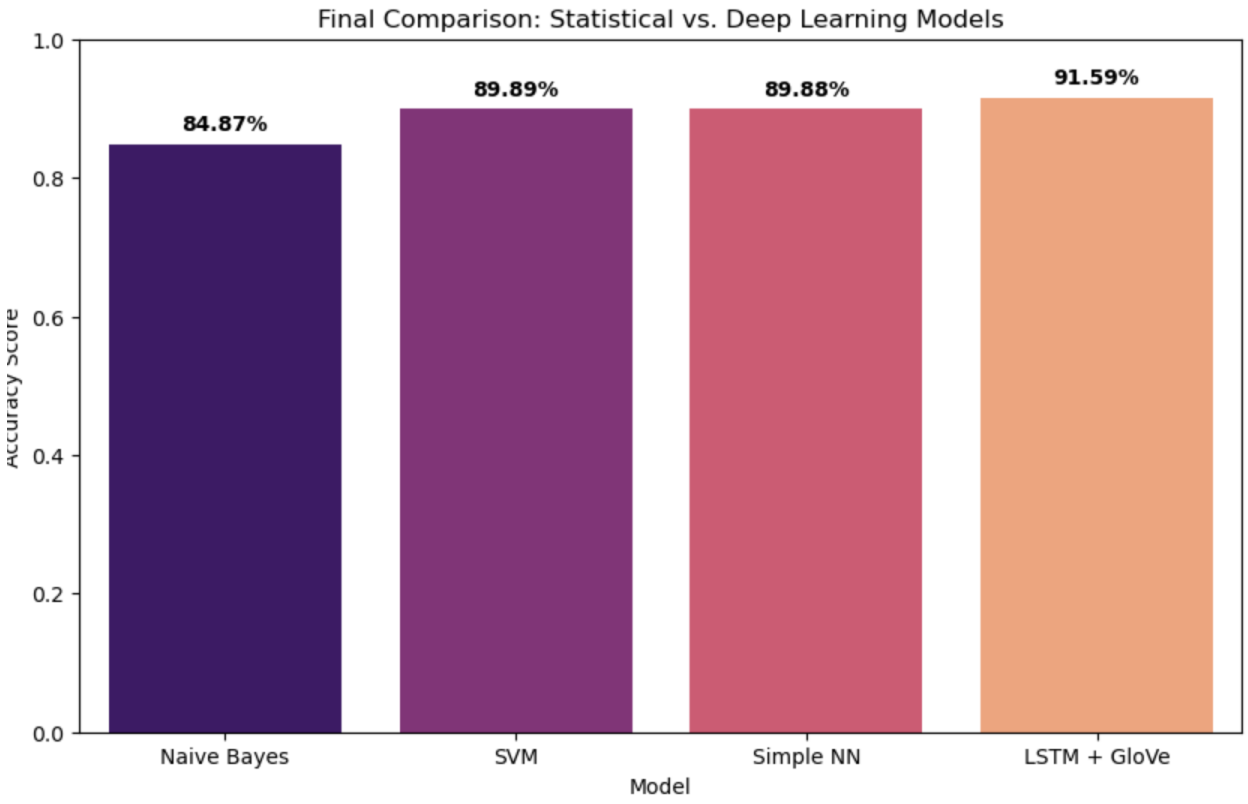
# Plot everything
results_all = pd.DataFrame({
    'Model': ['Naive Bayes', 'SVM', 'Simple NN', 'LSTM + GloVe'],
    'Accuracy': [acc_nb, acc_svm, acc_simple_dl, acc_glove_dl]
})

plt.figure(figsize=(10, 6))
sns.barplot(x='Model', y='Accuracy', data=results_all, palette='magma')
plt.ylim(0, 1)
plt.title('Final Comparison: Statistical vs. Deep Learning Models')
plt.ylabel('Accuracy Score')

for index, row in results_all.iterrows():
    plt.text(index, row.Accuracy + 0.02, f"{row.Accuracy:.2%}",
ha='center', fontweight='bold')

plt.show()
```

```
--- FINAL PROJECT COMPARISON ---
Naive Bayes Accuracy:      84.87%
SVM Accuracy:              89.89%
Simple Neural Net Acc:     89.88%
GloVe + LSTM Accuracy:     91.59%
```



```
print("--- VISUALIZING MODEL TRAINING HISTORY ---")

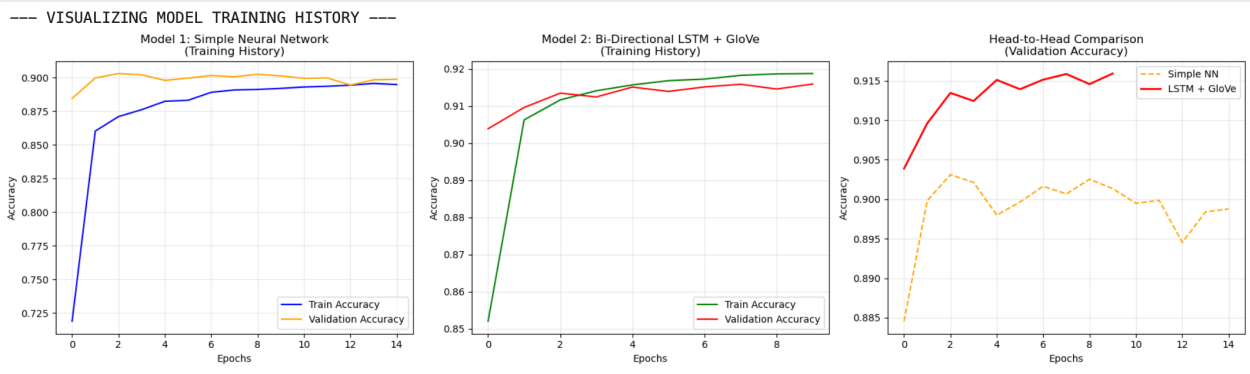
plt.figure(figsize=(18, 5))

# GRAPH 1: Simple Deep Learning Model History
# (This uses the 'history' variable from the first DL model)
plt.subplot(1, 3, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy',
color='blue')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy',
color='orange')
plt.title('Model 1: Simple Neural Network\n(Training History)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

# GRAPH 2: LSTM + GloVe Model History
# (This uses the 'history_glove' variable from new LSTM model)
plt.subplot(1, 3, 2)
plt.plot(history_glove.history['accuracy'], label='Train Accuracy',
color='green')
plt.plot(history_glove.history['val_accuracy'], label='Validation
Accuracy', color='red')
plt.title('Model 2: Bi-Directional LSTM + GloVe\n(Training History)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

# GRAPH 3: Direct Comparison (Validation Only)
# This answers: "Which model learned better on unseen data?"
plt.subplot(1, 3, 3)
plt.plot(history.history['val_accuracy'], label='Simple NN',
color='orange', linestyle='--')
plt.plot(history_glove.history['val_accuracy'], label='LSTM + GloVe',
color='red', linewidth=2)
plt.title('Head-to-Head Comparison\n(Validation Accuracy)')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



```
print("\n---COMPARISON GRAPHS ---")


# ------------------------------
# Helper: safe rolling mean (no pandas needed)
# ------------------------------
def rolling_mean(x, w=5):
    x = np.asarray(x, dtype=float)
    if len(x) < w:
        return x
    return np.convolve(x, np.ones(w)/w, mode="valid")

# Grab histories (shorter names)
acc_1 = history.history["accuracy"]
val_acc_1 = history.history["val_accuracy"]
loss_1 = history.history.get("loss", None)
val_loss_1 = history.history.get("val_loss", None)

acc_2 = history_glove.history["accuracy"]
val_acc_2 = history_glove.history["val_accuracy"]
loss_2 = history_glove.history.get("loss", None)
val_loss_2 = history_glove.history.get("val_loss", None)

epochs_1 = np.arange(1, len(acc_1) + 1)
epochs_2 = np.arange(1, len(acc_2) + 1)

# ============================================================
# FIGURE A: Loss curves + head-to-head val_loss
# (shows optimization difficulty + overfitting in a way accuracy can't)
# ============================================================
plt.figure(figsize=(18, 5))

# A1: Simple NN loss history
plt.subplot(1, 3, 1)
if loss_1 is not None and val_loss_1 is not None:
    plt.plot(loss_1, label="Train Loss", color="blue")
    plt.plot(val_loss_1, label="Val Loss", color="orange")
    plt.title("Model 1: Simple NN\n(Loss History)")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True, alpha=0.3)
else:
    plt.text(0.5, 0.5, "Loss not found in 'history'.\n(Ensure model was
compiled with a loss.)",
             ha="center", va="center")
    plt.axis("off")

# A2: LSTM+GloVe loss history
plt.subplot(1, 3, 2)
if loss_2 is not None and val_loss_2 is not None:
    plt.plot(loss_2, label="Train Loss", color="green")
    plt.plot(val_loss_2, label="Val Loss", color="red")
    plt.title("Model 2: BiLSTM + GloVe\n(Loss History)")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True, alpha=0.3)
else:
```

```
    plt.text(0.5, 0.5, "Loss not found in 'history_glove'.\n(Ensure model
was compiled with a loss.)",
             ha="center", va="center")
    plt.axis("off")

# A3: Direct comparison (val_loss only)
plt.subplot(1, 3, 3)
if val_loss_1 is not None and val_loss_2 is not None:
    plt.plot(val_loss_1, label="Simple NN (Val Loss)", color="orange",
linestyle="--")
    plt.plot(val_loss_2, label="LSTM+GloVe (Val Loss)", color="red",
linewidth=2)
    plt.title("Head-to-Head Comparison\n(Validation Loss)")
    plt.xlabel("Epochs")
    plt.ylabel("Val Loss")
    plt.legend()
    plt.grid(True, alpha=0.3)
else:
    plt.text(0.5, 0.5, "Val loss missing.\nCan't plot val_loss
comparison.", ha="center", va="center")
    plt.axis("off")

plt.tight_layout()
plt.show()


# ============================================================
# FIGURE B: Generalization gap over time
# (how much the model "cheats" on training vs generalizes)
# ============================================================
plt.figure(figsize=(18, 5))

# B1: Accuracy gap (train - val); closer to 0 is better
plt.subplot(1, 3, 1)
gap_acc_1 = np.array(acc_1) - np.array(val_acc_1)
gap_acc_2 = np.array(acc_2) - np.array(val_acc_2)
plt.plot(gap_acc_1, label="Simple NN gap", color="orange",
linestyle="--")
plt.plot(gap_acc_2, label="LSTM+GloVe gap", color="red", linewidth=2)
plt.axhline(0, color="gray", linewidth=1, alpha=0.6)
plt.title("Generalization Gap\n(Train Acc - Val Acc)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy Gap")
plt.legend()
plt.grid(True, alpha=0.3)

# B2: Loss gap (val - train); if this grows, overfitting is worsening
plt.subplot(1, 3, 2)
if loss_1 is not None and val_loss_1 is not None and loss_2 is not None
and val_loss_2 is not None:
    gap_loss_1 = np.array(val_loss_1) - np.array(loss_1)
    gap_loss_2 = np.array(val_loss_2) - np.array(loss_2)
    plt.plot(gap_loss_1, label="Simple NN gap", color="orange",
linestyle="--")
    plt.plot(gap_loss_2, label="LSTM+GloVe gap", color="red",
linewidth=2)
    plt.axhline(0, color="gray", linewidth=1, alpha=0.6)
    plt.title("Generalization Gap\n(Val Loss - Train Loss)")
    plt.xlabel("Epochs")
    plt.ylabel("Loss Gap")
```

```
    plt.legend()
    plt.grid(True, alpha=0.3)
else:
    plt.text(0.5, 0.5, "Loss data missing.\nCan't plot loss gap.",
ha="center", va="center")
    plt.axis("off")

# B3: Best epoch marker (val_accuracy peak) for each model
plt.subplot(1, 3, 3)
best_ep_1 = int(np.argmax(val_acc_1)) + 1
best_ep_2 = int(np.argmax(val_acc_2)) + 1
plt.plot(epochs_1, val_acc_1, label="Simple NN (val acc)",
color="orange", linestyle="--")
plt.plot(epochs_2, val_acc_2, label="LSTM+GloVe (val acc)", color="red",
linewidth=2)
plt.scatter([best_ep_1], [val_acc_1[best_ep_1 - 1]], color="orange")
plt.scatter([best_ep_2], [val_acc_2[best_ep_2 - 1]], color="red")
plt.title("Best Validation Epoch\n(Where each model peaks)")
plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()


# ============================================================
# FIGURE C: Training stability (smoothed val accuracy)
# (shows consistency; helps justify model choice beyond “one lucky peak”)
# ============================================================
plt.figure(figsize=(18, 5))

w = 5  # smoothing window (change to 3 or 7 if you want)
sm_1 = rolling_mean(val_acc_1, w=w)
sm_2 = rolling_mean(val_acc_2, w=w)

plt.subplot(1, 2, 1)
plt.plot(val_acc_1, label="Simple NN (raw)", color="orange", alpha=0.4)
plt.plot(val_acc_2, label="LSTM+GloVe (raw)", color="red", alpha=0.4)
plt.title("Validation Accuracy (Raw)")
plt.xlabel("Epochs")
plt.ylabel("Val Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(1, 2, 2)
plt.plot(np.arange(w, w + len(sm_1)), sm_1, label=f"Simple NN (smoothed
w={w})", color="orange", linestyle="--")
plt.plot(np.arange(w, w + len(sm_2)), sm_2, label=f"LSTM+GloVe (smoothed
w={w})", color="red", linewidth=2)
plt.title("Validation Accuracy (Smoothed)\n(Shows stability /
convergence)")
plt.xlabel("Epochs")
plt.ylabel("Smoothed Val Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
# ============================================================
# FIGURE D: Confusion matrices (side-by-side)
# (most "meaningful" comparison for classification: where each model
fails)
# ============================================================
from sklearn.metrics import confusion_matrix

# Try to find the true labels in encoded numeric form (0..5)
# (Your notebook typically has y_test_enc from LabelEncoder)
if "y_test_enc" in globals():
    y_true = np.array(y_test_enc)
elif "y_test_cat" in globals():
    y_true = np.argmax(np.array(y_test_cat), axis=1)
else:
    y_true = None

# Predict with both DL models (needs the same X_test input type you
trained on)
# Use X_test_safe if available (your notebook created it as TF tensor)
X_for_pred = None
if "X_test_safe" in globals():
    X_for_pred = X_test_safe
elif "X_test" in globals():
    X_for_pred = X_test

y_pred_1 = None
y_pred_2 = None

if X_for_pred is not None:
    try:
        p1 = model_dl.predict(X_for_pred, verbose=0)
        y_pred_1 = np.argmax(p1, axis=1)
    except Exception as e:
        print("Could not predict with model_dl:", e)

    try:
        p2 = model_glove.predict(X_for_pred, verbose=0)
        y_pred_2 = np.argmax(p2, axis=1)
    except Exception as e:
        print("Could not predict with model_glove:", e)

# Class names for nicer axes (LabelEncoder -> le.classes_)
if "le" in globals():
    class_names = list(le.classes_)
else:
    class_names = [str(i) for i in range(6)]

if y_true is not None and y_pred_1 is not None and y_pred_2 is not None:
    cm1 = confusion_matrix(y_true, y_pred_1)
    cm2 = confusion_matrix(y_true, y_pred_2)

    plt.figure(figsize=(14, 6))

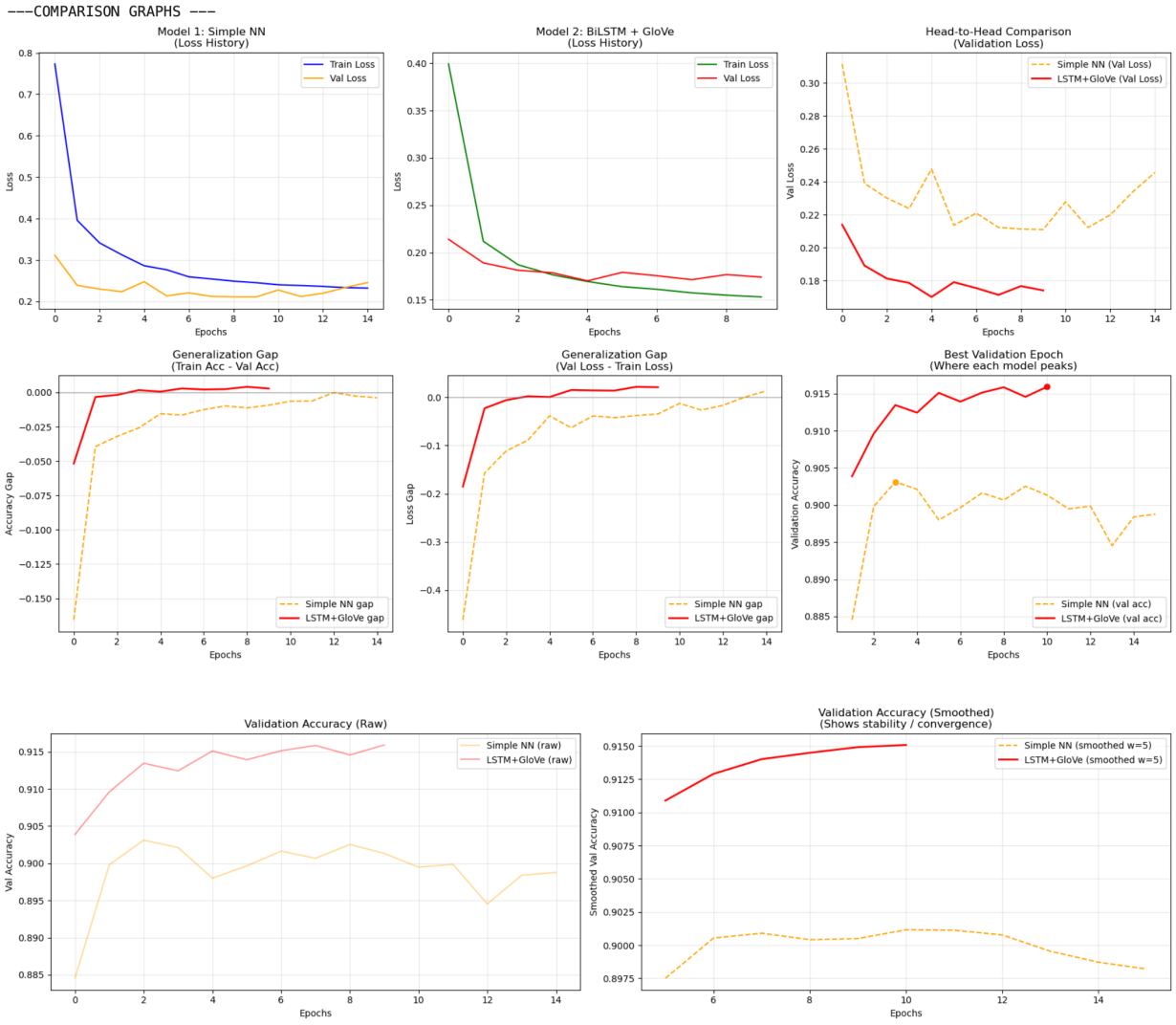
    plt.subplot(1, 2, 1)
    plt.imshow(cm1, interpolation="nearest")
    plt.title("Model 1: Simple NN\nConfusion Matrix")
    plt.xticks(np.arange(len(class_names)), class_names, rotation=45,
ha="right")
```

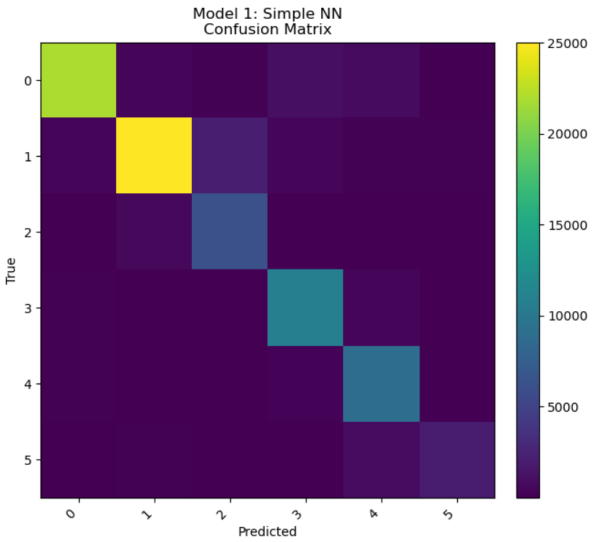
```
    plt.yticks(np.arange(len(class_names)), class_names)
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.colorbar(fraction=0.046, pad=0.04)

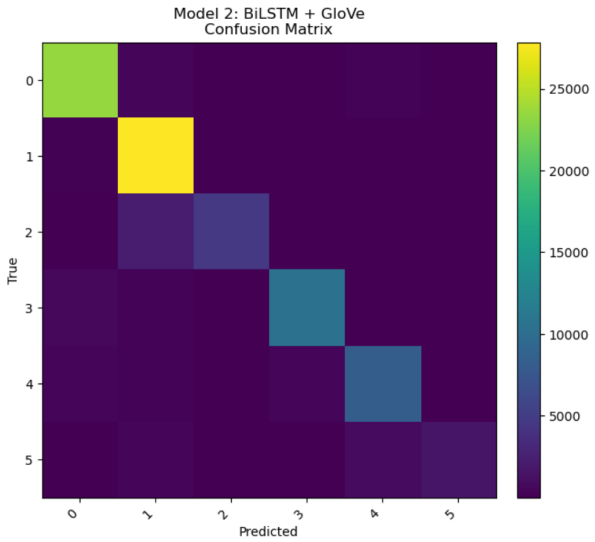
    plt.subplot(1, 2, 2)
    plt.imshow(cm2, interpolation="nearest")
    plt.title("Model 2: BiLSTM + GloVe\nConfusion Matrix")
    plt.xticks(np.arange(len(class_names)), class_names, rotation=45,
ha="right")
    plt.yticks(np.arange(len(class_names)), class_names)
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.colorbar(fraction=0.046, pad=0.04)

    plt.tight_layout()
    plt.show()
else:
    print("Skipping confusion matrices: missing y_true / predictions /
inputs.")
    print("Make sure y_test_enc (or y_test_cat), model_dl, model_glove,
and X_test_safe (or X_test) exist.")
```

```
---COMPARISON GRAPHS ---
```








```
print("\n--- EXTRA COMPARISON GRAPHS (VIOLIN + ECDF + CALIBRATION) ---")

# ------------------------------
# 1) Get y_true (numeric 0..5)
# ------------------------------
y_true = None
if "y_test_enc" in globals():
    y_true = np.array(y_test_enc)
elif "y_test_cat" in globals():
    y_true = np.argmax(np.array(y_test_cat), axis=1)

# ------------------------------
# 2) Choose X for prediction
# ------------------------------
X_for_pred = None
if "X_test_safe" in globals():
    X_for_pred = X_test_safe
elif "X_test" in globals():
    X_for_pred = X_test

# ------------------------------
# 3) Predict probabilities
# ------------------------------
p1 = p2 = None
if y_true is not None and X_for_pred is not None:
    try:
        p1 = model_dl.predict(X_for_pred, verbose=0)
    except Exception as e:
        print("Could not predict probs with model_dl:", e)

    try:
        p2 = model_glove.predict(X_for_pred, verbose=0)
    except Exception as e:
        print("Could not predict probs with model_glove:", e)

# Class names for nice labels
if "le" in globals():
    class_names = list(le.classes_)
else:
    class_names = [str(i) for i in range(6)]

if y_true is None or p1 is None or p2 is None:
    print("Skipping confidence-based plots: missing y_true and/or
probability predictions.")
```

```
else:
    # Predicted label + confidence per sample (max softmax)
    pred1 = np.argmax(p1, axis=1)
    pred2 = np.argmax(p2, axis=1)

    conf1 = np.max(p1, axis=1)
    conf2 = np.max(p2, axis=1)

    correct1 = (pred1 == y_true)
    correct2 = (pred2 == y_true)

    # ============================================================
    # GRAPH 1: VIOLIN PLOT — confidence distribution per TRUE class
    # - Shows which emotions the model is confident/uncertain on.
    # - SEE overlap + long tails (hard classes).
    # ============================================================
    data1 = [conf1[y_true == c] for c in range(len(class_names))]
    data2 = [conf2[y_true == c] for c in range(len(class_names))]

    plt.figure(figsize=(16, 6))

    ax1 = plt.subplot(1, 2, 1)
    ax1.violinplot(data1, showmeans=True, showmedians=True,
showextrema=True)
    ax1.set_title("Model 1: Simple NN\nConfidence by True Class
(Violin)")
    ax1.set_xlabel("True Class")
    ax1.set_ylabel("Predicted Confidence (max softmax)")
    ax1.set_xticks(np.arange(1, len(class_names) + 1))
    ax1.set_xticklabels(class_names, rotation=30, ha="right")
    ax1.grid(True, alpha=0.3)

    ax2 = plt.subplot(1, 2, 2)
    ax2.violinplot(data2, showmeans=True, showmedians=True,
showextrema=True)
    ax2.set_title("Model 2: BiLSTM + GloVe\nConfidence by True Class
(Violin)")
    ax2.set_xlabel("True Class")
    ax2.set_ylabel("Predicted Confidence (max softmax)")
    ax2.set_xticks(np.arange(1, len(class_names) + 1))
    ax2.set_xticklabels(class_names, rotation=30, ha="right")
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()


    # ============================================================
    # GRAPH 2: ECDF — confidence distributions for correct vs incorrect
    # - Shows if the model "knows when it's wrong".
    # - A better model has correct ECDF shifted to higher confidence,
    #   and incorrect ECDF shifted lower (separation).
    # ============================================================
    def ecdf(x):
        x = np.sort(np.asarray(x))
        y = np.arange(1, len(x) + 1) / len(x)
        return x, y

    plt.figure(figsize=(16, 6))
```

```
    plt.subplot(1, 2, 1)
    x_c, y_c = ecdf(conf1[correct1])
    x_w, y_w = ecdf(conf1[~correct1])
    plt.plot(x_c, y_c, label="Correct")
    plt.plot(x_w, y_w, label="Incorrect")
    plt.title("Model 1: Simple NN\nECDF of Confidence (Correct vs
Incorrect)")
    plt.xlabel("Confidence (max softmax)")
    plt.ylabel("ECDF")
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    x_c, y_c = ecdf(conf2[correct2])
    x_w, y_w = ecdf(conf2[~correct2])
    plt.plot(x_c, y_c, label="Correct")
    plt.plot(x_w, y_w, label="Incorrect")
    plt.title("Model 2: BiLSTM + GloVe\nECDF of Confidence (Correct vs
Incorrect)")
    plt.xlabel("Confidence (max softmax)")
    plt.ylabel("ECDF")
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()


    # ============================================================
    # GRAPH 3: CALIBRATION (Reliability diagram)
    # - If confidence=0.8, are you actually correct ~80% of the time?
    # - The closer the curve is to the diagonal, the better calibrated.
    # ============================================================
    def reliability_curve(conf, correct, n_bins=10):
        bins = np.linspace(0.0, 1.0, n_bins + 1)
        centers = []
        accs = []
        counts = []
        conf = np.asarray(conf)
        correct = np.asarray(correct, dtype=float)

        for i in range(n_bins):
            lo, hi = bins[i], bins[i + 1]
            m = (conf >= lo) & (conf < hi) if i < n_bins - 1 else (conf
>= lo) & (conf <= hi)
            if np.any(m):
                centers.append(conf[m].mean())
                accs.append(correct[m].mean())
                counts.append(m.sum())
        return np.array(centers), np.array(accs), np.array(counts)

    c1, a1, n1 = reliability_curve(conf1, correct1, n_bins=10)
    c2, a2, n2 = reliability_curve(conf2, correct2, n_bins=10)

    plt.figure(figsize=(8, 6))
    plt.plot([0, 1], [0, 1], linestyle="--", label="Perfect calibration")
    plt.plot(c1, a1, marker="o", label="Simple NN")
    plt.plot(c2, a2, marker="o", label="LSTM + GloVe")
    plt.title("Calibration Comparison (Reliability Diagram)")
    plt.xlabel("Mean Predicted Confidence")
```

```
plt.ylabel("Observed Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
--- EXTRA COMPARISON GRAPHS (VIOLIN + ECDF + CALIBRATION) ---
```

